SOC Verification with System Verilog

Ramdas M

Expert Verification Engineer

Content/Agenda

- Verification Concepts
  - What/Why/How
  - Simulation based and Formal
  - Assertions and Coverage based
  - Directed vs.. Random
- Introduction to System Verilog
  - History and evolution of SV
  - Comparison with Verilog
  - Language Features
  - OOP Concepts and support in SV
- Standard Verification Methodologies
  - Need for Methodologies
  - Layered Testbench Architecture
  - TLM concepts
  - OVM/UVM concepts

Acknowledgements

- I have reused lots of materials available over web along with my own customization based on experience to put it all together
- Thanks to following useful content
  - Accellera SystemVerilog workshop foils at DAC2003
  - Verification Academy Trainings and OVM/UVM cookbooks
  - Writing Testbenches using System Verilog – Janick Bergeron
  - *System Verilog lectures by Ahmed Hemani @ school of ICT, KTH*

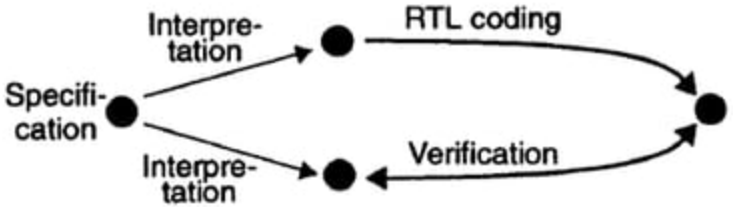
www.verificationexcellence.in

- For more Course Resources, Tutorials see
- **http://verificationexcellence.in/**

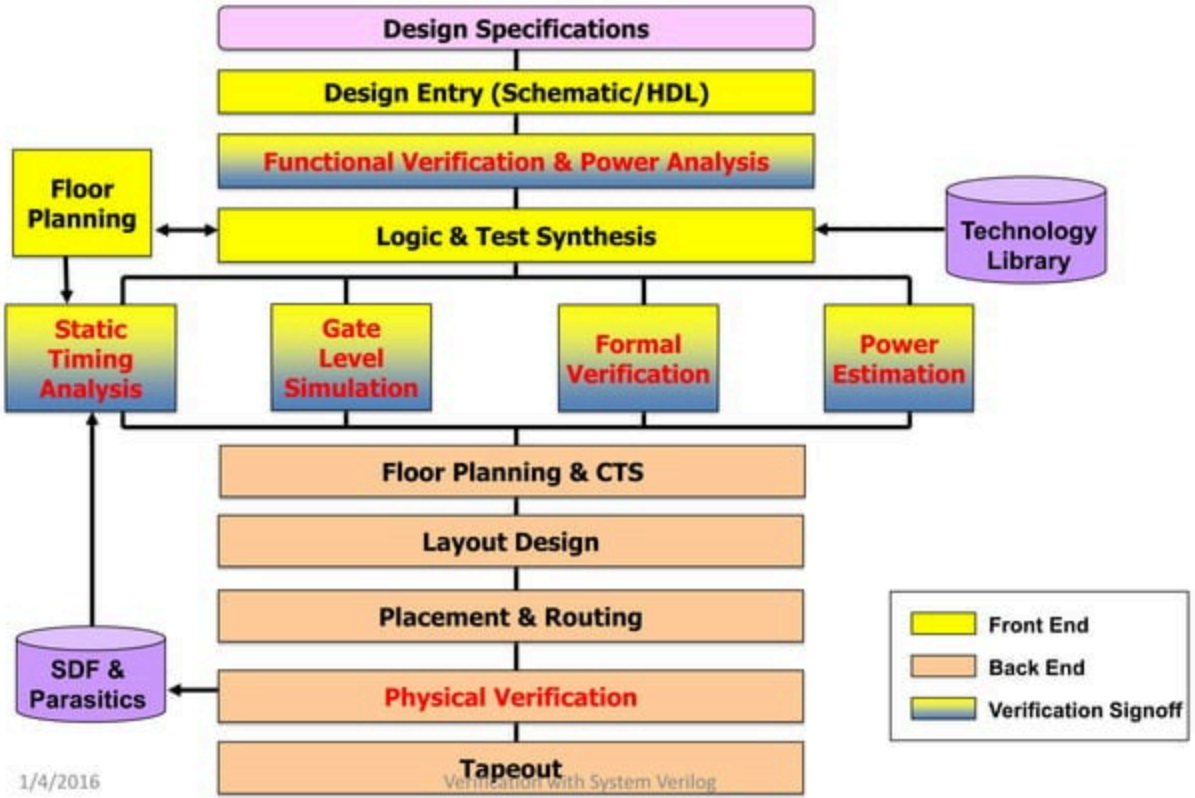
Verification Concepts

What is verification

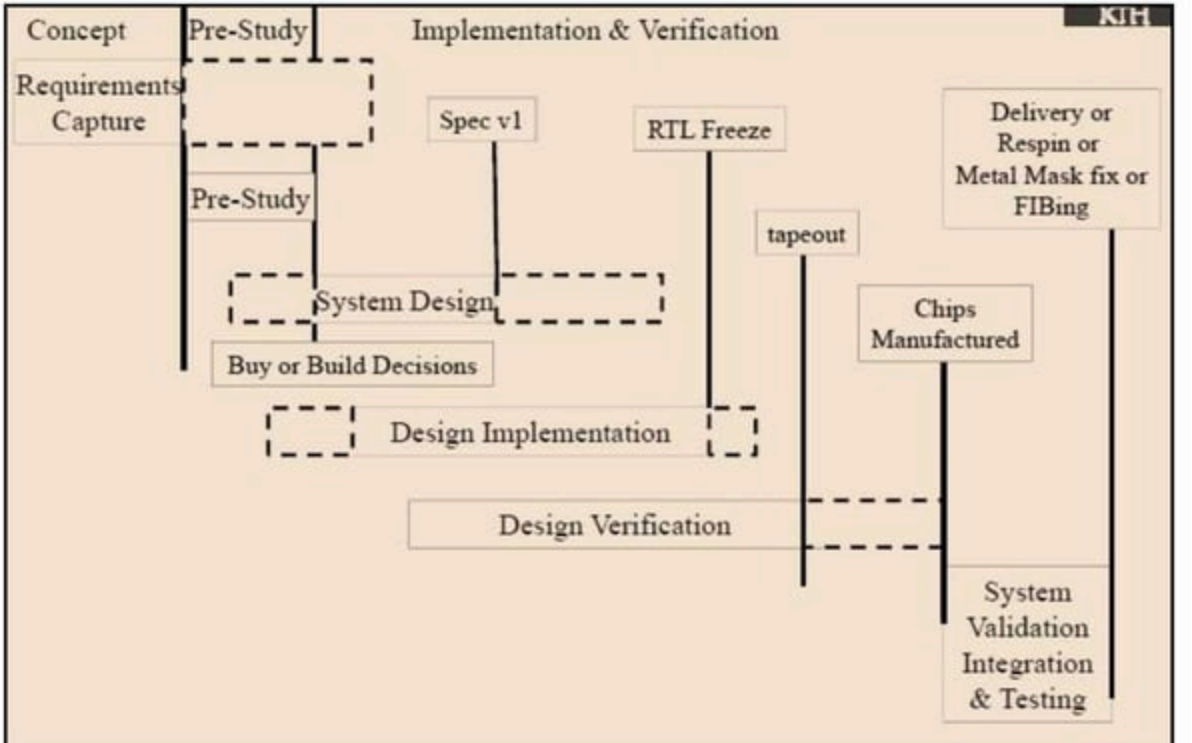
- Process of demonstrating functional correctness of a design.
- Process that ensures specification is preserved in the implementation.
- Different from validation and testing
  - Validation ensures that product/design satisfied user needs
  - Testing ensures that product is manufactured correctly



Verification in VLSI Design Flow



Verification in a SOC Project Life Cycle



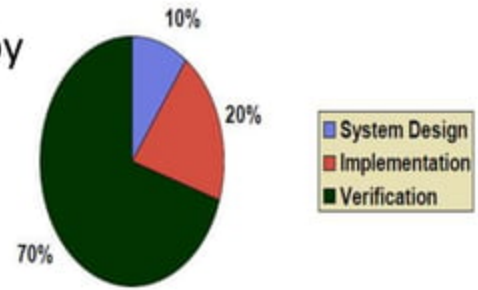
What if we don't verify?

- Is it possible to guarantee a design without verification?
  - Incorrect/Insufficient Specifications
  - Misinterpretation of specifications
  - Misunderstanding between designers
  - Missed cases
  - Protocol non-conformance

- Implications on the final system
  - Incorrect interaction between IPs/Cores
  - Conflict in accessing shared resources
  - Deadlock and arbitration
  - Priority conflicts in exception handling
  - **Unexpected behavior of the system**

The Importance of Verification

- 70% of design effort goes in verification
- Bug escapes to silicon can be costly including re-spin
  - 50% reason for re-spins are because of bugs. Hence the need to find all bugs early
- Ever increasing complexity of designs makes this harder
- Hence Verification is always on critical path for any product design
- Verification time can be reduced by
  - Parallelism
  - Abstraction
  - Automation



Verification Space

- What to Verify?
  - Functional Verification
  - Timing Verification
  - Performance Verification
- How to Verify?
  - Simulation based Verification
  - Emulation/FPGA based Verification
  - Formal Verification
  - Semi-Formal Verification
  - HW/SW Co- Verification

What is a Verification Plan?

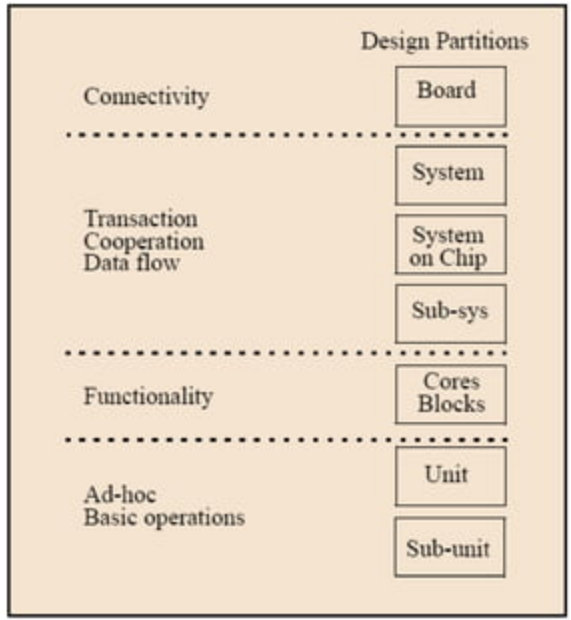
- Verification plan is the specification document for verification effort
  - Design specification is the golden reference
  - Mechanism to ensure all essential features are verified as needed
    - What to verify ?
      - Features and under what conditions to verify them
    - How to verify ?
      - What methodologies to use – Formal, Checking, Coverage etc.
      - What should be - Stimulus, Checkers, Coverage
    - Priority for features to be verified
  - Schedules should be extractable from a detailed plan

Verification Approaches

- Black-Box
  - ☺ Verification without knowledge of design implementation.
  - ☹ Lack of visibility and observability.
  - ☺ Tests are independent of implementation.
  - ☹ Impractical in todays designs.
- White Box
  - ☺ Intimate knowledge of design implementation
  - ☺ Full visibility and observability
  - ☹ Tests are tied to a specific implementation
- Grey Box
  - ☺ Compromise between above approaches

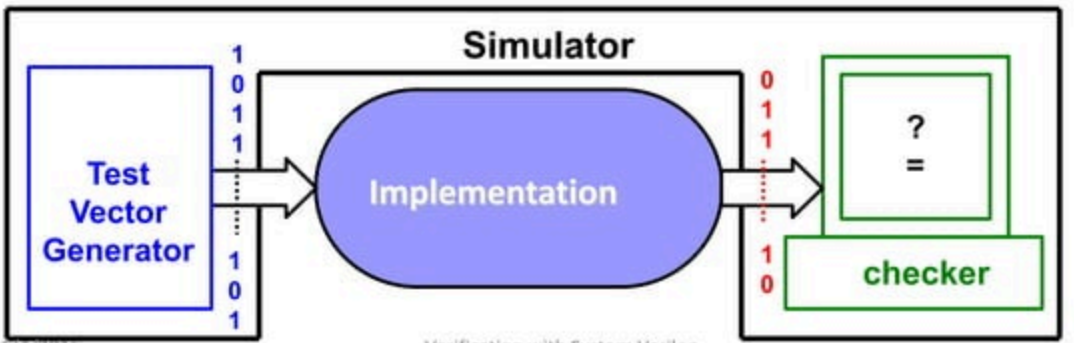
Levels of Verification

- Each level of Verification will be suited for a specific objective
  - Lower levels have better controllability and visibility
- Block level Verification helps designs to be verified independently and in parallel
- System Level Verification focuses more on interactions



Simulation Based Verification

- Generate Input sequences
  - Directed/Random/Constrained-Random sequences
- Generate expected output sequence (golden)
- Simulate the DUT with the generated input sequences
- Verify DUT output against golden output
- Use Coverage Metrics for ensuring completeness



Formal Verification

- Equivalence Checking
  - Checks for functional equivalence (not correctness) between RTL and netlist or between two models
- Model Checking
  - Algorithm takes two inputs
    - A finite state transition system (FSM) representing the implementation
    - A formal property representing the specification
  - The algorithm checks whether the FSM *"models"* the property correctly
    - This is an exhaustive search of the FSM to see whether it has any path or state that violates the property

Semi-Formal Verification

- Best of both worlds (Simulation & Formal)
- Use simulation to reach interesting states in the design
- Then fork off formal to do exhaustive analysis around the interesting state
- Finds bugs sitting deep in the design
- Useful for bug hunting
- Difficult to guide tool in proper direction
- Unlike pure formal, cannot give a PASS status

Assertion Based Verification

- An Assertion is a statement about a design's intended behavior ,which must be verified
- **Benefits of Assertions:**
  - Improving Observability and debug ability
  - Improving integration through correct usage checking
    - A design team inserts boundary assertions to monitor correct interface communication during integration verification
  - Improving verification efficiency
    - Find bugs faster
    - Work at all times and with all tools
    - Facilitate formal analysis
  - Improving communication through documentation
  - Useful in both static and dynamic simulations

Assertion Based Verification (2)

- Using Assertions in Formal Verification
  - Formally specify all functional requirements and behavior of design in a language
  - Use a procedure to prove that all specified properties hold true
- Using Assertions in Dynamic Simulation
  - Specified properties are useful as checks and for coverage

Coverage

- What is Coverage?
  - Coverage is the metric of completeness of verification
- Why coverage?
  - Verification is based on samples
    - Cannot run all possible tests (2^n) to cover full space
    - Need to know that all areas of the design have been verified
    - Quality of a test suite is not a measure of it's size
  - Solution:
    - Coverage Analysis

Coverage (2)

- Coverage Goals
  - Measure the quality of a test suite
  - Tell a verification engineer when to stop!
  - Achieve 100%
- Coverage Metrics
  - Systematically created list of verification tasks
  - Check that each task has been covered during verification
  - Supplement verification plan by pointing to untested areas
  - Help create/better regression suites
- Forms of Coverage
  - Structural (Code)
  - Functional coverage

Code Coverage

- Statement
  - Has each statement of the source code been executed?
- Branch
  - Has each control structure been evaluated to both true and false?
  - Example: if, case, while, repeat, forever, for, loop
- Condition
  - Has each boolean sub-expression evaluated both to true and false?
  - Extension of Branch coverage for conditional expressions

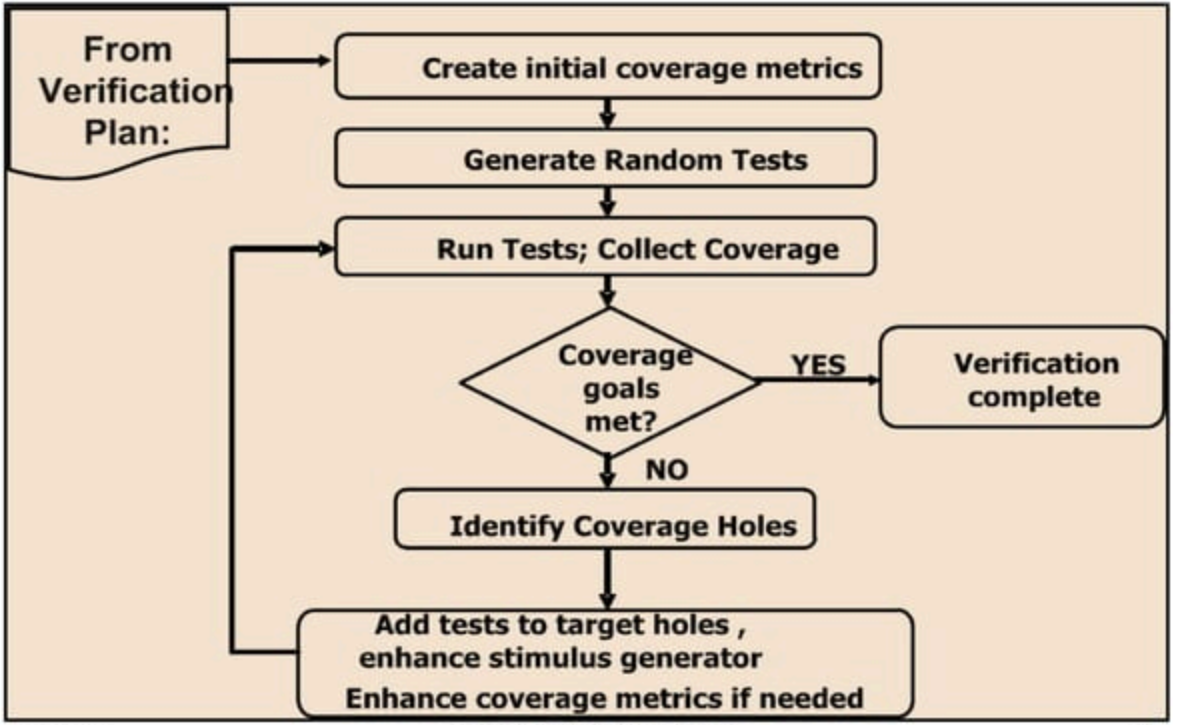
Code Coverage (2)

- Expression
  - Counts the no. of times an expression on the RHS of an assignment statement is executed
  - Example: **x <= a xor (not b);**
- Toggle
  - counts each time a logic node transitions from one state to another
  - Standard – covers 0->1 and 1->0 transitions
  - Extended – covers 0<->1, z<->1 and z<->0 transitions
- FSM
  - counts state reachability and transitions
  - State coverage and FSM Arc coverage

Functional Coverage

- Covers the functionality of the DUT
  - Define functional coverage metrics in the verification plan
- Functional Coverage is derived from design specification
  - DUT Inputs – Are all input operations injected?
  - DUT Outputs/functions – Are all responses seen from every output port?
  - DUT internals –
    - Are all interested design events being verified?
      - e.g. FIFO fulls, arbitration mechanisms etc.

Coverage Driven Verification



Coverage Analysis

Functional Coverage	Code Coverage	Interpretation
Low	Low	Still lot of verification to do
Low	High	Muti-cycle scenarios, corner cases, cross-correlations still to be covered
High	Low	Verification plan and/or functional coverage metrics inadequate. Possibility of duplicate/dead code
High	High	High confidence in verification quality

Directed Vs Random Testing

Directed

- ☹ Can only cover scenarios thought through planning
- ☹ High maintenance cost
- ☺ Works good when condition space is finite
- ☺ No need of extensive coverage coding

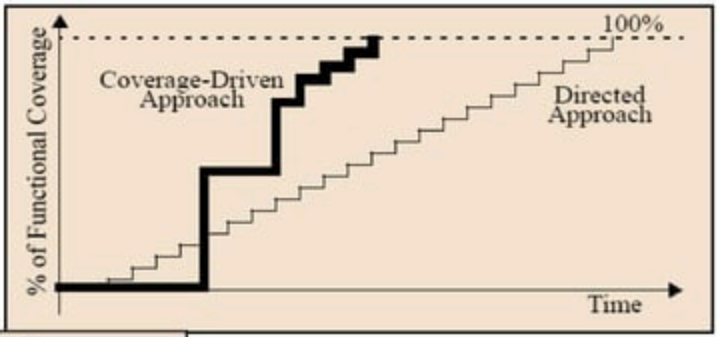
Constrained Random

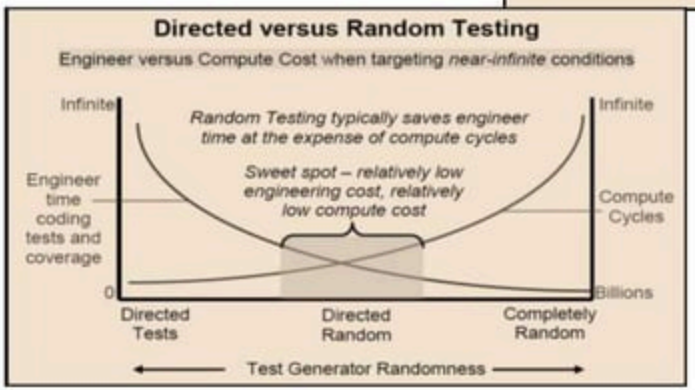
- ☹ High ramp up time to build smart test generators
- ☹ Need to identify and implement Functional Coverage
- ☺ Deep User control. Complex Test generator
- ☺ Best balance between engineer time and compute time
- ☺ Can have static and dynamic (during test run) randomness

Pure Random

- ☹ Need infinite compute cycles to cover all condition space
- ☺ Less user control. Simple to build generator

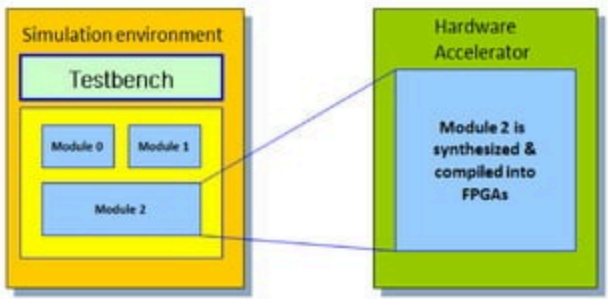
Directed vs. Random Testing





Hardware-Accelerated Simulation

- Simulation performance is improved by moving the time-consuming part of the design to hardware.
- Usually, the software simulation communicates with FPGA-based hardware accelerator
- Challenges
  - Generally improves speed but degrades on HW-SW (Testbench) communication
  - Abstracting HW-SW communication at transaction level rather than cycle level desired for better speeds

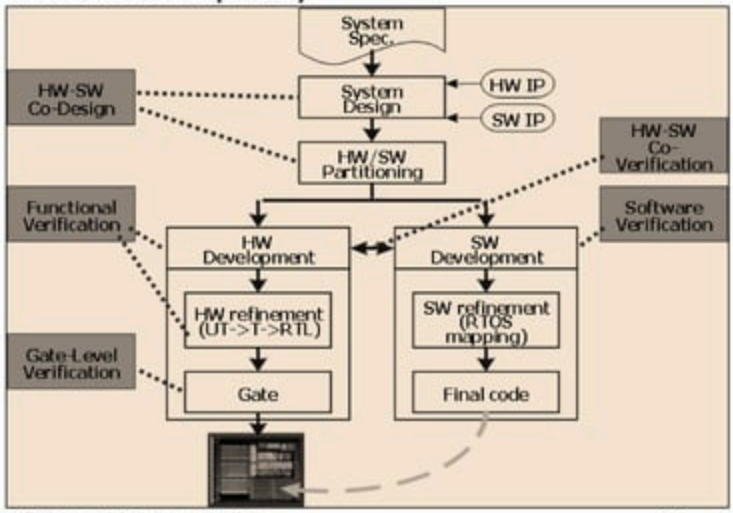


- HW Emulation
  - Full mapping of HW into an emulator (array of FPGAs)
  - More like a real target system. Speed up possible upto 1000X simulation
  - Debug is a challenge with limited visibility
  - Usually used for HW+SW co-verification

HW/SW Co-Verification

- SOC designs involve both HW and SW development
- HW and SW components are verified separately but needs to work together in real product
- Co-Verification will help projects to complete in short time and with higher confidence on both HW and SW quality

- Three parts for Co-Verification
  - Platform (FPGA, Emulator, ICE)
  - Stimulus (HW or SW stimulus, real Applications)
  - Debuggability (Trackers, Waveform captures, SW debuggers)



Verification Metrics

- Metrics
  - Measurements that provide visibility into a process
  - Help identify issues so corrective action can taken
  - Historical trend data facilitates future project planning
- Metrics can help us
  - Understand how the design was verified
  - Measure the productivity of various stimulus methods
  - Ensure that the desired checkers are in place and performing checks
  - Ensure regressions are run effective and productive
  - Coverage metrics helps in identifying stimulus holes
  - Bug metrics can provide insight into the simulation, coverage and overall progress
- Overall Metrics helps us tracking towards completeness and quality of verification

Examples of Various Verification Metrics

Process and Focused Areas	Process and Focused Area Attributes and the Information Associated Metrics Can Provide			
Design	Abstraction level	Simulated performance	List of instantiated blocks (and versions)	
Stimulus	Source of stimulus	Type of stimulus (CR, firmware, graph, legacy, etc.)		
Checking	Source of checkers	Results of checkers	Checker abstraction levels	
IP	Interface activity	Key internal states		
Coverage	Categories of coverage	RTL/stimulus/checker reference model	Abstraction level of coverage	
Build	Source and rev of files	Initial configuration used		
Run	Simulator/Emulator	Host machine info (memory, disk image distance, etc.)	Simulation performance	Revision of tools
Debugging	Area of failure	Commonality of cases where many tests report same failure		
Regress	Which simulations	Errors found	Errors re-found (i.e., wasted simulation)	Improvements in coverage results
Bug Status	Open bugs	Bug discovery info: stimulus, abstraction level, checker…	Metrics used to isolate bug	Bug closure information (sim time, engineer time, number of runs)

Introduction to System Verilog

www.verificationexcellence.in

- For more Course Resources, Tutorials see
- **http://verificationexcellence.in/**

What is System Verilog

- **SystemVerilog** is a combined hardware description language and hardware verification language
- SystemVerilog is an extensive set of enhancements to IEEE 1364 Verilog-2001 standards
- It has features inherited from Verilog HDL,VHDL,C,C++

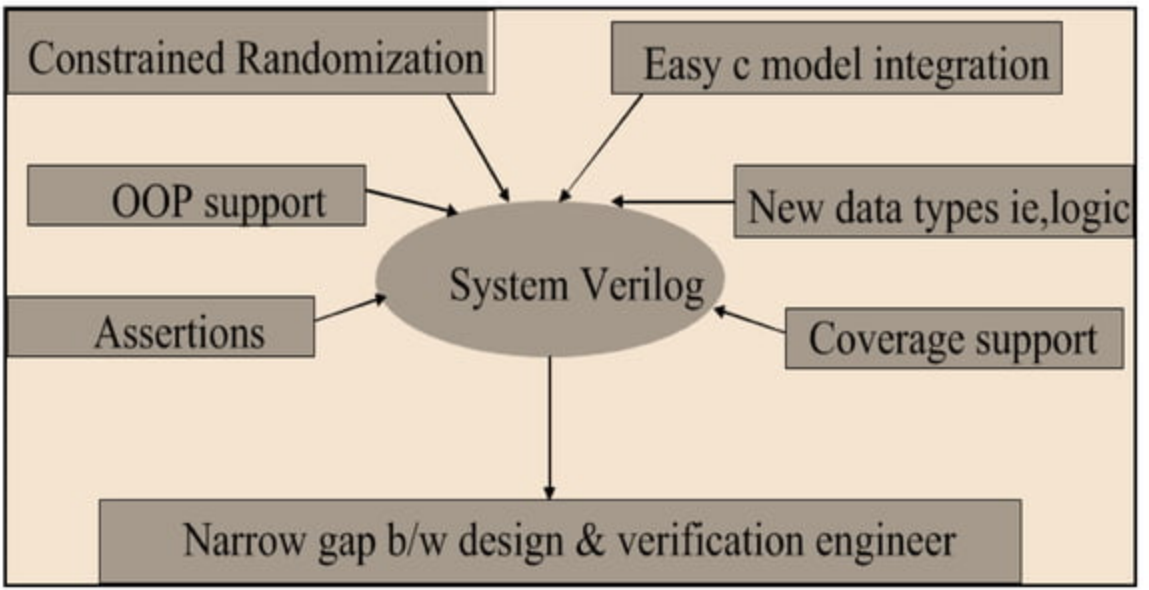
History and evolution of SV

- Verilog (IEEE standard 1364)
  - Began in 1983 as a proprietary language
  - Opened to the public in 1992
  - Became an IEEE standard in 1995 (updated in 2001 and 2005)
  - Between 1983 and 2005 design sizes increased dramatically!
- System Verilog (IEEE standard 1800)
  - Originally intended to be the 2005 update to Verilog
  - Contains hundreds of enhancements and extensions to Verilog
  - Published in 2005 as a separate document
  - Officially superseded Verilog in 2009
  - Updated with more features in 2012 (IEE 1800 2012 standard)

System Verilog – User View

- Has 5 major parts
  - SVD – System Verilog for Design
    - Features supporting Design
  - SVTB – System Verilog for Test benches
    - Test bench specific Features
  - SVA – System Verilog Assertions
    - Features for temporal and concurrent assertions
  - SVDPI – SV Direct Programming Interface
    - For better C/C++ integration
  - SVAPI – SV Application Programming Interface
    - For better Coverage/Assertion integration

Why System Verilog



Verilog Comparison

Verilog

- Used for Design Entry
- Module Level Verification

System Verilog

- Module Level Design
- Constrained Random Verification
- Assertions/Coverage
- One single language for entire design flow

Verilog Comparison – Data types

Verilog

- Strict about usage of wire & reg data type
- Variable types are 4 state – 0,1,X,Z

System Verilog

- Logic data type can be used so no need to worry about reg & wire
- 2 state data type added – 0, 1 state
- 2 state variable can be used in test benches,where X,Z are not required
- 2 state variable in RTL model may enable simulators to be more efficient

Verilog Comparison

Verilog

- Used for Design Entry
- Module Level Verification

System Verilog

- Module Level Design
- Constrained Random Verification
- Assertions/Coverage
- One single language for entire design flow

Verilog Comparison

Verilog

- Used for Design Entry
- Module Level Verification

System Verilog

- Module Level Design
- Constrained Random Verification
- Assertions/Coverage
- One single language for entire design flow

Verification Capabilities

Verilog

- File I/o
- Random number generation
- Fork/join
- Initial block
- Task & functions
- PLI

System Verilog

- All Verilog features
- Constrained random number generation
- Classes and OOP features
- Fork/join_any,fork/join_none
- Final block
- Task & function enhancements
- DPI

Direct Programming Interface

- DPI is an interface between System Verilog and C that allows inter-language function calls
- Simple to used as compared to PLI's
- Values can be passed directly
- Import Functions
  - SV calls C functions
- Export Functions
  - C calls SV functions

Why not VHDL ?

- VHDL Lacks
  - Constrained Random Generation
  - Functional Coverage
  - Assertions
- Specman E/Vera
  - Used with VHDL and Verilog for Constrained Random generation and Functional Coverage
- PSL (Property Specification Language)
  - Used for Assertions
- Learning 1 language (System Verilog) is better than learning 2 languages (Specman E/Vera and PSL)

Data Types

2 valued data type

bit: *1 bit (packed array)*
byte: *8 bit (character)*
shortint: *16 bit*
int: *32 bit*

4 valued data type

logic: *1 bit (packed array)*

integer: *32 bit*
time: *64 bit unsigned*

```
bit [14:0] bus; // unsigned
bit signed [11:0] i,q;

shortint unsigned addrs;
logic [15:0] addrs;
```

```
string myName = "John Smith";
byte c = "A"; // assign to c "A"
bit [1:4][7:0] s = "hello" ; // assigns to s "ello"
```

Integer Data Type

shortint	2-state (1, 0), 16 bit signed
int	2-state, 32 bit signed
longint	2-state, 64 bit signed
byte	2-state, 8 bit signed
bit	2-state, user-defined vector size
logic	4-state (1,0,x,z) user-def
reg	4-state user-defined size
integer	4-state, 32 bit signed
time	4-state, 64 bit unsigned

Signed/Unsigned

- byte, shortint, int, integer and longint defaults to signed
  - Use unsigned to represent unsigned integer value

    Example: **int unsigned ui;**
- bit, reg and logic defaults to unsigned
- To create vectors, use the following syntax:

  logic [1:0] L; // Creates 2 bit logic vector

Strings

- String – dynamic allocated array of bytes
- SV provides methods for working with strings

Str1 == Str2	Equality
Str1 != Str2	Inequality
<, <=, >, >=	Comparison
{Str1, Str2, … Strn}	Concatenation
Str1[index]	indexing – return 0 if out of range

String Methods

- `len`
- `putc`
- `getc`
- `toupper`
- `tolower`
- `compare`
- `icompare`
- `substr`

- `atoi, atohex, atoct, atobin`
- `atoreal`
- `itoa`
- `hextoa`
- `octtoa`
- `bintoa`
- `realtoa`

Operators

Bitwise Operators		
~	~m	Invert each bit of m
&	m&n	Bitwise AND of m and n
|	m|n	Bitwise OR of m and n
^	m^n	Bitwise EXOR of m and n
~^	m~^n	Bitwise EX NOR of m and n

Unary Reduction Operators		
&	&m	AND all bits of m together (1-bit result)
~&	~&m	NAND all bits of m together (1-bit result)
|	|m	OR all bits of m together (1-bit result)
~|	~|m	NOR all bits of m together (1-bit result)
^	^m	EXOR all bits of m together (1-bit result)
~^	~^m	EXNOR all bits of m together (1-bit result)

Source/Acknowledgement: On-line Verilog HDL Quick Reference Guide

Operators

Arithmetic Operators		
+	m+n	Add n to m
-	m-n	Subtract n from m
-	-m	Negate m (2's complement)
*	m*n	Multiply m by n
/	m/n	Divide m by n
%	m%n	Modulus of m/n

Logical Operators		
!	!n	Is m not true?
&&	m&&n	Are both m AND n true?
||	m||n	Are both m OR n true?

Operators

Equality Operators (Compares logic values of 0 and 1)		
==	m==n	Is m equal to n? (1-bit True/False result)
!=	m!=n	Is m not equal to n? (1-bit True/False result)
Identity Operators (Compares logic values of 0,1,X and Z)		
===	m^n	Is m identical to n? (1-bit True/False results)
!==	m~^n	Is m not equal to n? (1-bit True/False result)
Wild Eqality Operators		
=?=	m=?=n	A equals b, X and Z values are wild cards
!?=	m!?=n	A not equals b, X and Z values are wild cards

- The three types of equality (and inequality) operators in SystemVerilog behave differently when their operands contain unknown values (X or Z).
- The == and != operators result in X if any of their operands contains an X or Z.
- The === and !== check the 4-state explicitly, therefore, X and Z values shall either match or mismatch, never resulting in X.
- The =?= and !?= operators treat X or Z as wild cards that match any value, thus, they too never result in X.

SV additions for Flow Control

- Verilog includes: **if-(else-(if)), case, forever, repeat, while, for, ?: (ternary)**
- System Verilog:
  - Enhances for
    - Loop variable declaration
    - Multiple statements in init and step blocks (comma separated)
    - ++ and -- operators (Mentioned in operator section)
  - Adds do..while, foreach

```
for (int i; i < arr.size(); j+=2,
    i++) begin
    arr[i] += 200;
      arrb[i]--;
 end
```

```
1)x = 0;
  while (x)  begin
     $display("%d", x);
     x--;
  end
```

```
2) do
    begin
    $display("%d", x);
     x--;
    end
    while (x);
```

case/casez/casex

- **case:** 4-value exact matching, runtime evaluation, no fall through, bit length of all expressions padded to same length
- **casex:**
  - Handles wildcard
  - 'z' or 'x' bits will both match
- **casez:**
  - Handles wildcard
  - 'z' bit in either item or expression will be treated as a match for that bit

forever

- Continuous execution, without end, of body statement(s)
- Used with timing controls
- Usually last statement in some block

```
initial : clock_drive
begin
  clk = 1'b0;
  forever #10 clk = ~clk;
end : clock_drive
```

repeat

- Repeat a block ‘x’ times, no conditional test
  - **repeat (*expr*) *statement***
- What happens with expr = ‘x’ or ‘z’?

- Example

```
x = 0;
repeat (16)
  begin
   $display("%d", x++);
  end
```

User defined and Enumerated types

Syntax:

typedef <base_data_type> <type_identifier>

eg: typedef int inch ; *// inch becomes a new type*

inch foot = 12, yard = 36; *// these are 2 new variables of type 'inch'*

Syntax:

enum [enum_base_type] { enum_name_declaration {,enum_name_declaration} }

enum_base_type: default is **int**

Enumeration is a useful way of defining abstract variables.

Define an enumeration with " enum "

enum {red, green, yellow} traf_lite1, traf_lite2;

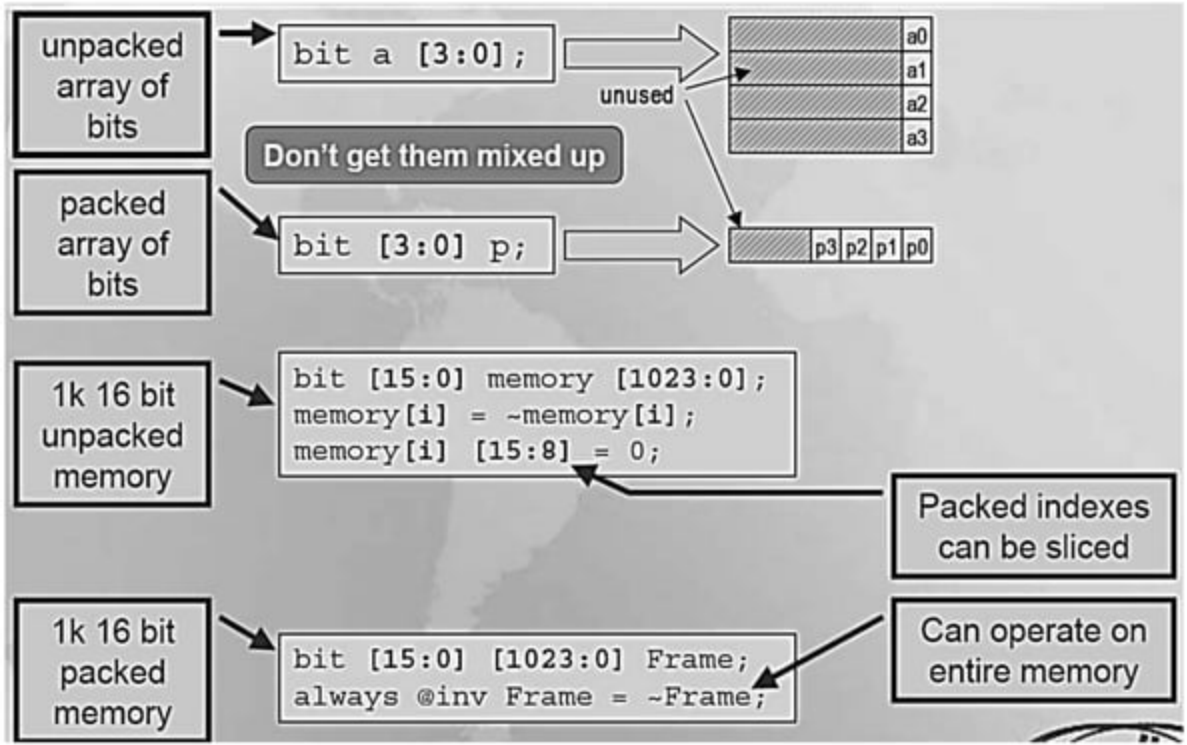
Values can be cast to integer types and auto-incremented

enum { a=5, b, c} vars; *// b=6, c=7*

A sized constant can be used to set size of the type

enum bit [3:0] { bronze=4'h3, silver, gold} medal; *// All medal members are 4-bits*

Packed and Unpacked Arrays



Dynamic Arrays

Dynamic declaration of one index of an unpacked array

Syntax:

```
data_type array_name[] ;
```

Declares a dynamic array *array_name* of type *data_type*

```
data_type array_name[] = new[ array_size ] [(array)] ;
```

Allocates a new array *array_name* of type *data_type* and size *array_size*
Optionally assigns values of *array* to *array_name*
If no value is assigned then element has default value of *data_type*

Examples:

```
bit [3:0] nibble[ ];          // Dynamic array of 4-bit vectors
integer mem[ ];               // Dynamic array of integers

int data[ ];                  // Declare a dynamic array
data = new[256];              // Create a 256-element array
int addr = new[100];          // Create a 100-element array
addr = new[200](addr);        // Create a 200-element array
                              // preserving previous values in lower 100 addresses
```

Dynamic Array - Methods

- **Resizing**

```
<array> = new[<size>](<src_array>);
dyn= new[j * 2](fix);
```

function int size()

Returns the current size of the array

```
int addr[ ] = new[256];
int j = addr.size(); // j =  256
```

function void delete()

Empties array contents and zero-sizes it

```
int addr[ ] = new[256];
addr.delete();
```

Associative Arrays

- Associative arrays are used when the size of the array is not known or the data is sparse.
- ***Syntax:***

  data_type array_name [index_type];

  In other words

  value_type array_name [key_type];
- It implements a lookup table of the elements of its declared type.
- Data type used as an index serves as lookup key and imposes an order.
- Associative array do not have their storage allocated until it is used.

Associative Array Methods

Function	Use
num()	Returns number of entries
delete(<index>)	Index for delete optional. When specified used to delete given index else whole array.
exists (<index>)	Returns 1 if element exists at index else 0
first (<index>), last (<index>)	assigns to the given index variable the value of the first/last (smallest/largest) index in the associative array. It returns 0 if the array is empty, and 1 otherwise.
next (<index>), prev (<index>)	finds the entry whose index is greater/smaller than the given index.

Queues and Lists

SV has a built-in list mechanism which is ideal for queues, stacks, etc.
A list is basically a variable size array of any SV data type.

```
int q1[$];              // $ represents the 'upper' array boundary
int n, item;

q1 = '{ n, q1 };        // uses concatenate syntax to write n to the left end of q1
q1 = '{ q1, n };        // uses concatenate syntax to write n to the right end of q1

item = q1[0];           // read leftmost ( first ) item from list
item = q1[$];           // read rightmost ( last ) item from list

n = q1.size;            // determine number of items on q1

q1 = q1[1:$];           // delete leftmost ( first ) item of q1
q1 = q1[0:$-1];         // delete rightmost ( last ) item of q1

for (int i=0; i < q1.size; i++)    // step through a list using integers (NO POINTERS)
  begin ... end

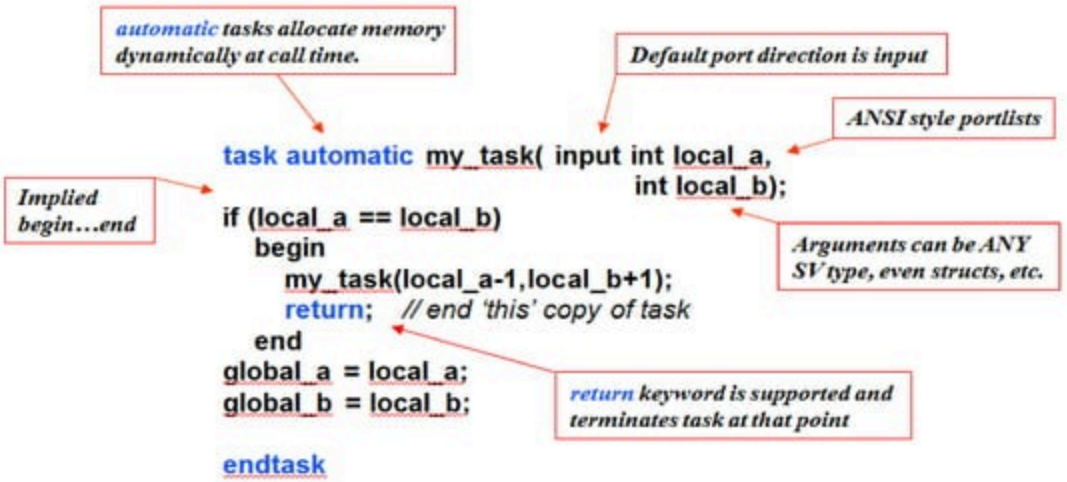
q1 = { };                          // clear the q1 list
```

Queue Methods

size()	Returns the number of items in the queue. If the queue is empty, it returns 0.	Prototype: function int size();
insert()	Inserts the given item at the specified index position. Q.insert (i, e) => Q = '{Q[0:i-1], e, Q[i,$]}	Prototype: function void insert (int index, queue_type item);
delete()	Deletes the item at the specified index position. Q.delete (i) => Q = '{Q[0:i-1], Q[i+1,$]}	Prototype: function void delete (int index);
pop_front()	Removes and returns the first element of the queue. e = Q.pop_front () => e = Q[0]; Q = Q[1,$]	Prototype: function queue_type pop_front();
pop_back()	Removes and returns the last element of the queue. e = Q.pop_back () => e = Q[$]; Q = Q[0,$-1]	Prototype: function queue_type pop_back();
push_front()	Inserts the given element at the front of the queue. Q.push_front (e) => Q = '{e, Q}	Prototype: function void push_front (queue_type item);
push_back()	Inserts the given element at the end of the queue. Q.push_back (e) => Q = '{Q, e}	Prototype: function void push_back (queue_type item);

Tasks and Functions

SystemVerilog makes a number of extensions to basic Verilog syntax.



Full recursion is supported (automatic variables/arguments stored on stack)

- Can do concurrent calls
- Can do recursive calls

Functions

automatic functions allocate memory dynamically at call time (full recursion).

Default port direction is input (also supports output)

ANSI style portlists

```
function automatic int factorial (int n);
  if (n==0) return(1); // factorial 0 is 1
  else return(factorial(n-1)*n);
endfunction
```

Implied begin…end

Arguments and return type can be ANY SV type, even complex structs, etc.

return(value) is supported and terminates function at that point

```
function void inverta();
  a = !a
endfunction

reg a;

initial
  inverta(); // function called like a task
```

Return type of void means no return value!

Recommended style (instead of writing a task) to guarantee a task executes with 0 delay.

Tasks and Functions - Usage

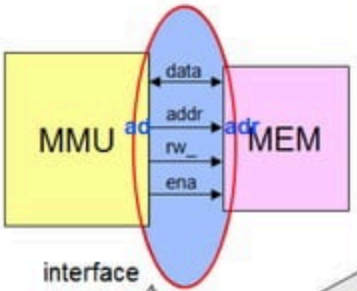
- Tasks
  - Tasks can enable other tasks and functions
  - Tasks may execute in non-zero simulation time.
  - Tasks may have zero or more arguments of type input, output and inout.
- Functions
  - Function can enable other functions only. Task cannot be called from functions.
  - Functions should execute in zero simulation time.
  - Functions have only one return value but System Verilog also allows functions to have input, output or inout types.
- Both tasks and functions support passing arguments by reference. By default arguments are passed by value.

Argument passing

- Passing by value is the default mechanism for passing arguments

Interfaces

Great coding efficiency can be achieved by modeling the blocks of a system at different levels of abstraction, behavioral, rtl, gate, etc.
In Verilog, the I/O between these blocks has always remained at the lowest "wire" level. High-performance system-level simulation requires the abstraction of inter-block communication.



At it's simplest an interface is like a module for ports/wires

```
module mmu(d, a, rw_, en);
  output [15:0] a;
  output rw_, en;
  inout [7:0] d;
  . . .
endmodule

module mem(d, a, rw_, en);
  input [15:0] a;
  input rw_, en;
  inout [7:0] d;
  . . .
endmodule

module system;
  wire [7:0] data;
  wire [15:0] addr;
  wire ena, rw_;

  mmu U1 (data, addr, rw_, ena);
  mem U2 (data, addr, rw_, ena);
endmodule
```

Traditional Verilog

```
interface interf;
  logic [7:0] data;
  logic [15:0] addr;
  logic ena, rw_;
endinterface

module mmu(interf io);
  io.addr <= ad;
  . . .
endmodule

module mem(interf io);
   adr = io.addr;
 . . .
endmodule

module system;
  interf i1;
  mmu U1 (i1);
  mem U2 (i1);
endmodule
```

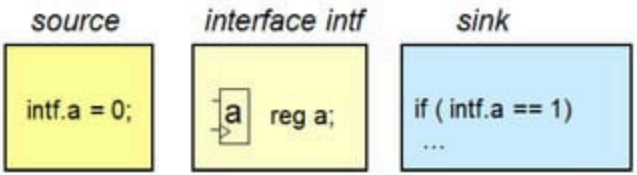
SystemVerilog

Abstracting IO



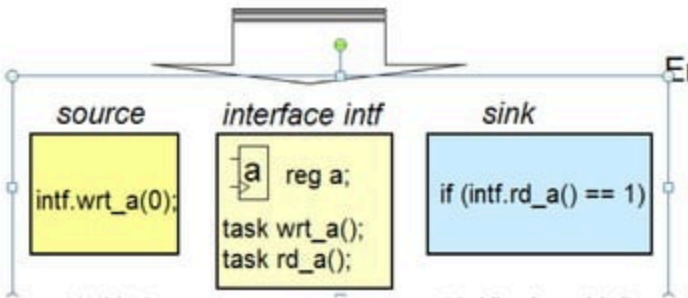
Traditional Verilog approach

- Simple netlist-level IO
- source/sink can be abstracted but IO must stay at low level
- IO operations are cumbersome



Simple "bundle" interface

- All accesses are through interface
- Simplifies source/sink declarations



Enhanced interface with methods

- source/sink only call methods
- source/sink don't see low-level "details" like variables/structure, etc.
- Easy to swap interface abstractions without any effect on source/sink

Interfaces – Characteristics

- Interfaces bring abstraction-level enhancements to ports, not just internals.
- An interface may contain any legal System Verilog code except module definitions and/or instances. This includes tasks, functions, initial/always blocks, parameters, etc.
- Bus timing, pipelining, etc. may be captured in an interface rather than the connecting modules.
- Interfaces are defined once and used widely, so it simplifies design.
  e.g. Changing a bus spec (add a new signal?) means editing the interface only.
- Interfaces are synthesizable.

Modports

- Different users of interface need different views
  - Master/Slave
- Restrict access to internal interface signals
  - Protect implementation signals from corruption

Example

```
interface i2;
 wire a, b, c, d;
 modport master (input a, b,
                 output c, d);
 modport slave  (output a, b,
                 input c, d);
endinterface : i2

module m (i2.master infm);
 ...
endmodule: m

module s (i2.slave infs);
 ...
endmodule: s
```

Clocking Blocks

- The clocking block assembles signals that are synchronous to a clock and makes their timing explicit.
- Clocking blocks capture timing and synchronization requirements from the structural, procedural and functional elements of a testbench.
- Enables writing of operations such as synchronous events, input sampling etc in a concise and precise manner

Input and output skews

- Input and output signals are sampled at a designated clocking event.
- For an input skew, the signal is sampled at skew units before the clock event.
- For an output or inout, the signal is driven simulation time units after the corresponding clock.
- A skew value is a constant expression and can be specified by a parameter.
- If a number is used for the time then skew is interpreted based on timescale.
- An input skew of 1step indicates that the signal is sampled at the end of previous time step.

```
clocking dram @(clk);
    input #1ps address;
    input #5 output #6 data;
endclocking

interface Bus (input bit clk,
    input bit req,
    output bit ack);

    clocking cb @(posedge clk);
        input clk;
        input req;
        output #4 ack;
    endclocking: cb
    modport master (clocking cb);
endinterface: Bus
```

Clocking block syntax

```
clocking block_name clocking_event;
    item list;
endclocking : block_name

clocking bus @(posedge clock1);
  default input #10ns output #2ns;
  input data, ready,
          enable=top.mem1.enable;
  output negedge ack;
  input #1 addr;
endclocking
```

Program Blocks

- A program block is similar to a module. It is used for testbench code.

```
program helloWorld();
  initial
  begin: hello
    $display("Hello World");
  end

  initial
  begin: there
    $display("Hello There");
  end
endprogram: helloWorld
```

Program Blocks

- Programs can be instantiated inside modules, but not the other way around.
- Program blocks may contain one or more initial blocks, but may not contain always, UDPs, modules, interfaces, or other programs.
- Programs may be explicitly exited using the $exit task.
- When all program blocks complete the simulation ends.
- Operations of a program block happen in the reactive region of the simulation.

Program block semantics

- Statements and constructs within a program block that are sensitive to changes (e.g., update events) on design signals are scheduled in the Reactive region.
- @(clk) S1;
  where clk is a design signal. Every transition of signal clk will cause the statement S1 to be scheduled into the Reactive region. The continuous assignment assign tclk = clk; would also be scheduled in the Reactive region.
- Initial procedures within program blocks are scheduled in the Reactive region.
- The standard # delay operator within program blocks schedules process resumption in the
- Reactive region.

Packages

- Creates a scope that contains declarations intended to be shared among one or more compilation units, modules, interfaces, or programs.
  - Generally constants, type defs, tasks and functions
  - No hierarchical references allowed

OOP Concepts

www.verificationexcellence.in

- For more Course Resources, Tutorials see
- http://verificationexcellence.in/

What is OOP?

- OOP is object oriented programming
  - Organize programs in same way as objects are organized in real world
  - Break program into blocks that work together to accomplish a task, each block has a well defined interface
- Class – A blueprint for a house
  - Program element "containing" related group of features and functionalities
  - Encapsulates functionality
  - Provides a template for building objects

Class Basics

- A *Class* is a description of some group of *things* that have something in common.
- Objects are individual instances of "classes".
  - Example: A class might be "Automobile". Instances of the "Automobile" class might be "Joe's car", "Bob's car", "Sally's truck", etc.
- Objects/Classes have:
  - Data
    - Color, speed, direction, etc.
  - Operations/Methods
    - Start, stop, increaseSpeed, turn, etc.
- Encapsulation:
  - Encapsulate implementation details internal to the object/class.

Classes

- **Inheritance**: (is-a relationship)
  - Allows users to extend existing classes, making minor modifications. Extending the "Automobile" class example, users might create subclasses for "sedan", "truck", "van", etc. The "van" class might also have a "minivan" subclass. Etc. In these cases, the subclass *IS-A* superclass. i.e. a "sedan" is a "Automobile".
  - When using inheritance, the sub-class "inherits" all the parents public/protected data properties and methods. It is allowed to override them, or use them as-is.
- **Composition:** (has-a relationship)
  - Composition is used for the case where one object *HAS-A* instance of another class. For example, an "Automobile" class might have 4 instances of a "wheel" class. In this case, a wheel is not an "Automobile", so inheritance should not be used.

Classes

- **Polymorphism:**
  - Most common definition of polymorphism is the ability of the language to process objects differently depending on their *data type* or *class*. System Verilog can only process objects differently depending on their class.
  - The primary usage of polymorphism in industry (object-oriented programming theory) is the ability of objects belonging to different types to respond to method, field, or property calls of the same name, each one according to an appropriate type-specific behavior.
  - The program does not have to know the exact type of the object in advance, and so the exact behavior is determined at run time (this is called late binding or dynamic binding).

Overloading vs. overriding

- Don't confuse the concepts of overloading and overriding
- Overloading
  - deals with multiple methods in the same class with the same name but different signatures
  - lets you define a similar operation in different ways for different data
- Overriding
  - deals with two methods, one in a parent class and one in a child class, that have the same signature
  - lets you define a similar operation in different ways for different object types.
- SystemVerilog supports overriding not overloading.

Class Definition

- **Definition Syntax**

  class*name*;
  <data_declarations>;
  <task/func_decls>;
  endclass

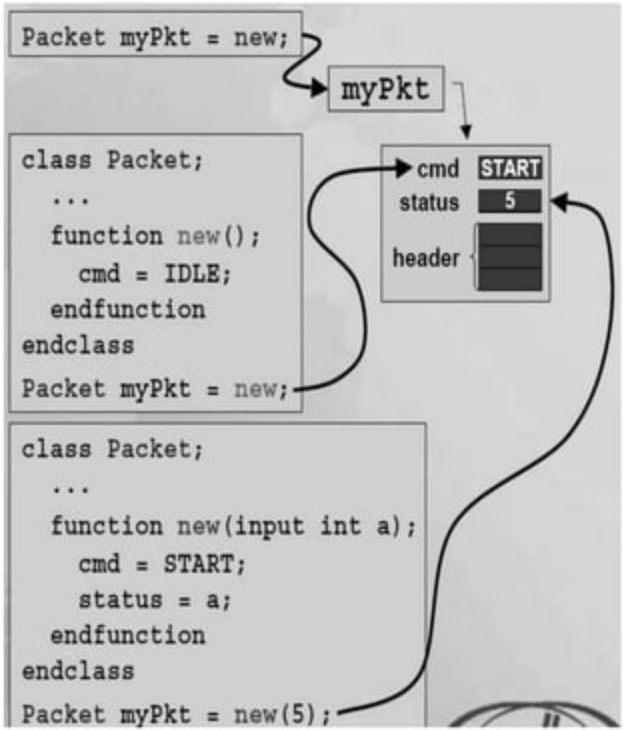
- Extern keyword allows out of body method declaration

```
class Packet;
  bit[3:0] cmd;
  int status;
  myStruct header;
  function int get_status();
    return(status);
  endfunction
  extern task set_cmd(input bit[3:0] a);
endclass
```

```
task Packet::set_cmd(input bit[3:0] a);
  cmd = a;
endtask
```

Class Instantiation

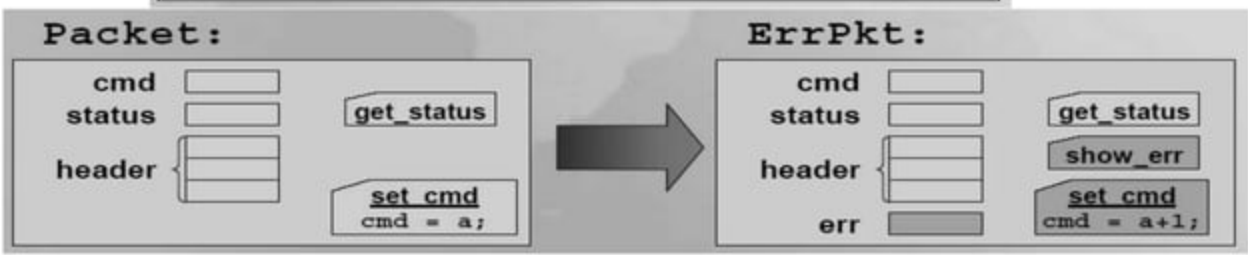
- Objects Allocated and Initialized Via Call to the **new** *Constructor* Method
- All objects have built-in **new** method
  - No args
  - Allocates storage for all data properties
- User-defined **new** method can initialize and/or do other things



Class Inheritance

- Keyword ***extends*** Denotes Hierarchy of Definitions
  - Subclass inherits properties and methods from parent
  - Subclass can redefine methods explicitly

```
class ErrPkt extends Packet;
  bit[3:0] err;
  function bit[3:0] show_err();
    return(err);
  endfunction
  task set_cmd(input bit[3:0] a);
    cmd = a+1;
  endtask // overrides Packet::set_cmd
endclass
```



Static properties

- Static properties/data members are "static" to all instances of the class. This means that all instances share the same value of this variable. If one instance changes the value, it changes the value for all instances

- Static methods do not require an instance of the class to operate on. Static methods may only modify static properties. To invoke a static method, use *Classname::methodName*

- The special variable ***this*** is a predefined object handle for the current object instance

Polymorphism usage

- Instances of subclasses may be assigned to variables declared of the superclass type.
- This is useful for cases where the general algorithm is the same for all the subclasses, but only a few details need to change.
- If the subclass overrides a method specified in the superclass, the method defined in the class of the object instance is called.

```
class BaseClass;
   virtual function in myFunc(int b);
        return(b + 10);
   endfunction myFunc
endclass: BaseClass
class myFirstClass extends BaseClass;
   virtual function int myFunc(int b);
        return(b - 3);
   endfunction: myFunc
endclass: myFirstClass
class mySecondClass extends BaseClass;
   virtual function int myFunc(int b);
        return(b + 3);
   endfunction: myFunc
endclass: mySecondClass

BaseClass bc;

// Returns an instance myFirstClass
bc = getFirstClassInstance();
$display("What do I print?  %d"
                  ,bc.myFunc(6));

// Returns an instance mySecondClass
bc = getSecondClassInstance();
$display("What do I print?  %d"
                  ,bc.myFunc(6));
```

Data Hiding and Encapsulation

- To make data members visible only to the class, use the *local* keyword.

```
class myPacket extends BasePacket;
  local int x;
```

- To make data members visible only to the class, or any subclasses, use the *protected* keyword.

```
class myPacket extends BasePacket;
  protected int x;
```

Constant Class Properties

- The *const* keyword may be used to make class properties unchangeable. *const* constants are different from `define constants because the initial value may be determined at runtime, and may be different per class instance.

```
class myPacket extends BasePacket;
  const int size; // Assignment of constant value in
                  //  declaration makes it constant
                  //  to all instances.

  function new(int id);
      size = id * 4096; // Single assignment in
                        // constructor OK

  ...
```

Abstract Classes

- The *virtual* keyword may be used on a class to make the class "abstract". An abstract class may not be instantiated. Users must subclass the abstract class to create instances of the class.

```
virtual class BasePacket;
  …
```

Typedef Class and Forward References

- Sometimes it is necessary to use a class before it has been defined. To do this, you can use a typedef forward reference, then later define the class. Example:

```
typedef class C2; // Forward declaration of C2
class C1;
  C2 c2Instance;
  …
endclass: C1
class C2;
  C1 c1Instance;
  …
endclass: C2
```

Parameterized Class

- Allows Generic Class to be Instantiated as Objects of Different Types
  - Uses module-like parameter passing
  - Avoids writing similar code more than once

```
class stack #(parameter type T = int;);
  local T items[];
  task push( T a ); ... endtask
  task pop( ref T a ); ... endtask
endclass

stack i_s; // default: a stack of int's

stack#(bit[1:10]) bs; // a stack of 10-bit vector

stack#(real) rs; // a stack of real numbers

stack#(Vfour) vs; // stack of classes
```

Virtual Interfaces

- Classes cannot have modules or interfaces, need a specialized mechanism
- Virtual interfaces provide a mechanism for separating test programs/BFM models from the actual signals.
- Virtual interfaces let BFM models manipulate virtual set of signals instead of actual RTL signals.
- Virtual interface is a variable that represents an interface instance.
- Syntax:
  virtual <interface name>
  <variable name>;

Virtual Interfaces Example

```
// interface
// definition
interface Bus
(input logic clk);
    bit req;
    bit grant;
    logic [7:0] addr;
    logic [7:0] data;
endinterface: Bus

// testbench
// interface instance
 Bus infc_b (clk);
// dut instance
 dut dut1 (infc_b, clk);
// class instance
 BFM mybfm =
             new
(infc_b);
```

```
 class BFM;
   virtual Bus bus;
   Xaction xaction;

   function new (virtual Bus b);
// need to initialize virtual interface
// in constructor
      bus = b;
      xaction = new;
   endfunction

   task req_bus();
     @(posedge bus.clk);
     bus.req <= 1'b1;
     $display("Req = %b @ %0t",
              bus.req, $time);
   endtask: req_bus
endclass: BFM
```

Random Constraints

Simplest randomness

- **$urandom** system tasks
- **$urandom()** is SV, thread stable, deterministic
- **$urandom** returns unsigned 32-bit integers
- Procedural call can be inserted wherever needed

Better Randomness

- Constraints are built onto the Class system
- Random variables use special modifier:
  - **rand** –random variable
  - **randc** –random cyclic variable
- Object is randomized by calling **randomize( )**method
  - Automatically available for classes with random variables.
- User-definable methods
  - **pre_randomize()**
  - **post_randomize()**

Constraints

- Set of **Boolean** algebraic expressions
- Relationships between random variables and:
  - Other random variables
  - Non-random state variables

- Example:
- **constraint a_le_b { a <= b; }**
- **constraint c_eq_10 {c == 10;}**
- **constraint b_in_range { b >= 2 && b <= 8; }**
- **constraint all_gt_0 {a > 0; b > 0; c > 0;}**

Conflicting constraints

- What happens when you impose constraints that conflict in some way?
- Example:
  - **constraint x_gt_y { x > y; }**
  - **constraint y_gt_z { y > z; }**
  - **constraint z_gt_x { z > x; }**
  - **…**
  - **if (x_inst.randomize() == 0) // Solver error**
  - **begin**
  - **…**
  - **end**

Constraint operators

- Any Verilog boolean expression

  i.e. **x < y+b-c*10>>20**

- Other constraint operations
  - set membership — **within**
  - implication — **->** or **if…else…**
  - iterative constraint — **foreach**
  - variable ordering — **solve … before**
  - functions — **func_x()**

Implication constraint

- Uses one boolean to decide if another constraint must hold

```
(trans_size == SMALL) -> (length < 10)
(trans_size == MED) ->
     (length >= 10 && length <= 100)
(trans_size == LARGE) -> (length > 100)
```

- Advanced note:

  `a -> b` *is equivalent to* **`!a || b`**

Loop/array constraints

- Constrain every element of an array in some fashion, including reference to other elements of array

```
constraint c1 {
   foreach ( A[i] )
      A[i] inside {2, 4, 6, 8, 10};
   }
constraint c2 {
   foreach ( A[k] )
      (k < A.size-1) -> A[k+1] > A[k];
   }
```

Distribution Constraints

- Operators: **:= :/ dist**
- Example:

```
constraint twsConstraint
{
  tws dist
  {
    [0:2]    :/ 10,
    [3:9]    :/ 1,
    [10:50]  :/ 9
  };
}
```

```
constraint distConstraint
{
  cmd dist
  {
    mem_write := 10,
    mem_read  := 5,
    lrw       := 1,
    lrrww     := 1,
    io_read   := 1,
    io_write  := 1,
    idle      := 1
  };
}
```

std::randomize()

- Procedural invocation of constraint solver
- Any variables can be the random variables
- “with” block for constraints
- Normal .randomize() cannot include variables outside scope of class

Arguments to .randomize()

- Normal form of **.randomize()**
  - No arguments, randomize class members according to declaration modifiers
- Optional arguments
  - Specify the variables which are random
  - Can make 'rand' override declaration type for this call
  - Declares entire set of random variables for this run of Constraint Solver
  - 'null' argument forces checking constraints only
  - Cannot change '**randc**' to '**rand**'

Inline Constraints

- Specify inline constraints that are added to constraint set to solve

```
trans.randomize() with { x < 100;  z > buzz; };
```

Constraint Guards

- Guards prevent the application of constraints, similar to implication, but remove the constraint from consideration of the constraint solver
- Use the same implication operator, just using state variables

```
(!global_reset_state && cmd) -> (cmd_type != IDLE)
```

- Can even test obj pointers for null

```
(a != null && a.x > 10) ->
        (y < 100 && y > a.x)
```

Enable/Disable rand/constraints

- rand_mode – method to toggle the “rand” attribute off on a class variable
- constraint_mode – method to toggle the application of a named constraint

Layered Constraints

- Constraints Inherited via Class Extension
  - Just like data and methods, constraints can be inherited or overridden

```
typedef enum{ low, high, other }AddrType;
class MyBusextends Bus;
rand AddrTypetype;
constraint addr_rang {
   ( type == low ) =>addrin { [ 0 : 15] };
   ( type == high ) =>addrin { [128 : 255] }; }
endclass
```

- **Bus::word_align** Constraint is also active
  - Inheritance allows layered constraints
  - Constraints can be enabled/disabled via **constraint_mode()**method

A Full Example

```
module test_rand;
typedef enum {SM, MED, LRG} trans_len;
class transaction {
  rand bit [19:0] addr;
  rand trans_len len;
  rand bit [3:0] byte_len;
  rand bit wr_or_rd_b;

  constraint c1 { wr_or_rd_b -> len != LRG; }
  constraint c2 { (len == SM)  -> (byte_len <= 4); }
  constraint c3 { (len == MED) -> (byte_len >  4 && byte_len <=
   8);}
  constraint c4 { (len == LRG) -> (byte_len >  8); }
  // Mem above 16-bit is write-only IO devices
  constraint c5 { (!wr_or_rd_b) -> (addr[19:16] != 0); }

  virtual function string toString()
    return sformat("%5H: %s %d bytes", addr, (wr_or_rd_b ? "WR" :
   "RD"), byte_len);
  endfunction : toString
endclass : transaction
<CONTINUED>
```

A Full Example (2)

```
transaction trans = new();
intial
  begin : test_body
    int counter = 0;
    repeat (20)
      begin : rand_gen
        if (trans.randomize() == 0)
          begin
            $display("ERROR: Random constraints
   conflict");
            $finish;
          end
       $display("%d: %s", counter++, trans.toString());
       end : rand_gen
  end : test_body

endmodule : test_rand
```

Process/Threads

Sequential vs.. Concurrent Blocks

- Sequential Block:
  - Simulator executes statements in a sequential block in sequence
  - It finishes the current statement, then begins the next
  - You always know the order in which it actually executes the statements
  - The simulator exits the block after finishing the last statement
- Concurrent Block
  - Simulator executes statements in a concurrent block in parallel
  - It starts executing all statements simultaneously
  - You can not know the order in which it actually executes statements scheduled for the same simulation time
  - The simulator exits the block after finishing the latest statement.
  - A **return** statement in the context of fork..join is illegal.

Process and Threads

- Threads are created via fork…join
- Threads execute until a blocking statement
  - wait for: (event, mailbox, semaphore, variable, etc.)
- $exit terminates the main program thread

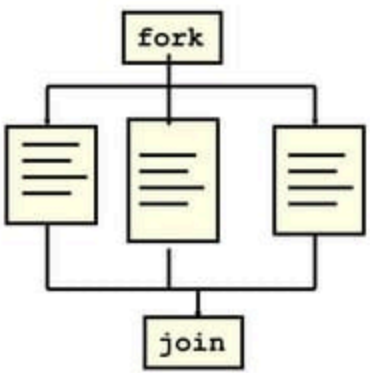
- **Multiple Independent Threads Maximize Stimulus Interactions**

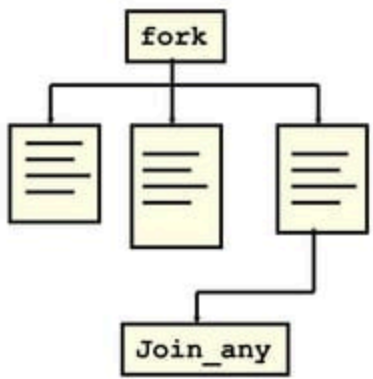
fork-join	fork-join_any	fork-join_none

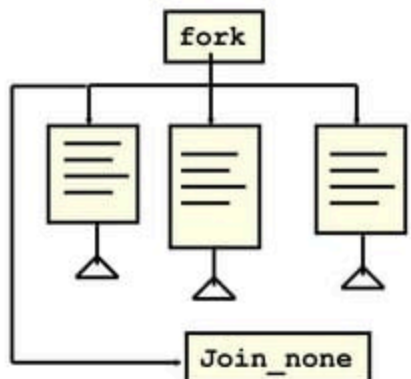
```
fork
   a;b;c;
join
```

```
fork
   a;b;c;
join_any
disable fork //kill all child
                // process
wait fork   //wait till child
            //process over
```

```
fork
   a;b;c;
join_none
```



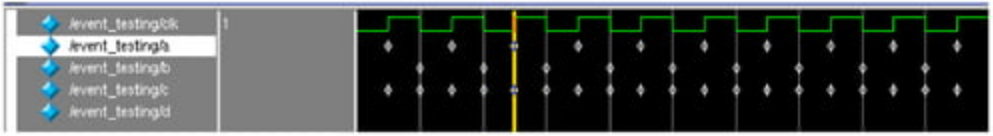




Events Trigger Types [1]

- **Triggering an event**
  - Named events are triggered using -> operator.
  - Triggering an event unblocks all processes currently waiting on the event.
  - These events are such that trigger state cannot be observed but only their effect.
- Event can be visualized in wave window.

```
module event_testing ();
  event a, b;
  bit clk;
  always @(posedge clk)
      -> a;
  always @(negedge clk)
      -> b;
  initial
    begin
        clk = 1'b0;
        forever #10 clk = !clk ;
    end
endmodule
```



Events Trigger Types [2]

- **Nonblocking event trigger**
  - They are triggered using ->> operator.
  - The statement executes without blocking and it creates a nonblocking assign update event in the time in which the event occurs.
  - The effect of this event is felt during the nonblocking assignment region of a simulation cycle.

Example

```
always @(posedge clk)
begin
  if (counter == 2)
     ->> a;
  counter++;
end

initial
begin
 forever @(a)
   $display("event a
     triggered @ %0t, $time);
end
```

Waiting for an event

- @ is used to wait for an event.
- The @ operator blocks the calling process until the given event is triggered.

Example

```
module event_testing ();
  event a, b, c;
  bit clk;
  always @(posedge clk)
      -> a;
  always @(negedge clk)
      -> b;
  always @(a or b)
      -> c;
  initial
    begin
     clk = 1'b0;
     forever #10 clk = !clk ;
    end
endmodule
```

Event Variables [1]

- **Merging Events**
  - When one event variable is assigned to another, both merge into one event variable.
  - Executing -> on either one of the events affects processes waiting on either event variable.

Example

```
event a, b;
a = b;
-> a; // also triggers b
-> b; // also triggers a
```

Event Variables [2]

- **Event Comparison**
  - Event variables can be compared against other event variables or the special value null.
  - Equality (==) with another event or with null.
  - Inequality (!=) with another event or with null.

Example

```
event E1, E2;
if ( E1 )
// same as if ( E1 != null )
  E1 = E2;
if ( E1 == E2 )
  $display( "E1 and E2 are the
             same event" );
```

Semaphores

- Semaphores are typically used for mutual exclusion, access control to shared resources, and basic synchronization
- Can be described as counters used to control access to shared resources by multiple processes [threads].
  - eg: Accessing a shared bus
- Implemented as a bucket with fixed number of keys
  - Processes using semaphores must first procure a key before executing
  - All others must wait until a sufficient number of keys is returned to the bucket.

Semaphore Methods

- Semaphore provides following built-in methods:

Method	Use
new()	Create a semaphore with specified number of keys.
put()	Return one or more keys back.
get()	Obtain one or more keys.
try_get()	Try to get one or more keys without blocking.

Semaphore example

```
module semaphore_test ();
 semaphore spr = new(2);

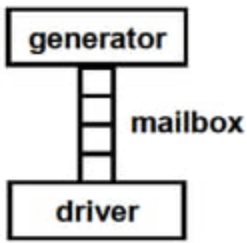
 initial
 begin:init1
  #1 spr.get(1);
  $display("initial1 takes
     1 key at %0t", $time);
  #6 spr.put(1);
  $display("initial1 returns
       1 key at %0t",$time);
  #1 spr.get(1);
  $display("initial1 takes 1
        key at %0t", $time);
 end
 initial
 begin:init2
  #5 spr.get(2);
  $display(" inital2 takes 2
        keys at %0t",$time);
  #5 spr.put(1);
  $display(" inital2 returns
       1 key at %0t",$time);
 end
endmodule: semaphore_test
```

Output:

```
# initial1 takes 1 key at 1
# initial1 returns 1 key at 7
#  inital2 takes 2 keys at 7
#  inital2 returns 1 key at 12
# initial1 takes 1 key at 12
#  q -f
```

Mailboxes

- Mailbox is a communication mechanism that allows messages to be exchanged between different processes.
  - Data can be sent to a mailbox by one process and retrieved by another



Mailbox Types

- Mailboxes can be classified as:
  - **Unbounded mailboxes**
    - No restrictions placed on size of mailbox.
    - put() will never block.
    - Ex: **mailbox m = new ();**
  - **Bounded mailboxes**
    - Number of entries is determined when the mailbox is created.
    - Bound value should be positive.
    - put() will be blocked if the mailbox is full.
    - Ex: **mailbox m = new (5);** // mailbox of depth = 5

Mailbox Methods

- Messages are placed in strict FIFO order. This does not guarantee order of arrival but that the arrival order shall be preserved.
- Mailboxes provides following built-in methods:

Method	Use
new()	Create a new mailbox.
put()	Place a message in a mailbox.
get()	Retrieve a message from mailbox.
try_get()/ try_peek()	Try to retrieve a message from the mailbox without blocking.
try_put()	Try to place a message in mailbox without blocking. Useful only for bounded mailboxes.
peek()	Copies a message from mailbox without actually removing it.

Mailbox example

```
module mailbox_ex ();
 class Xaction;
    rand bit [2:0] addr;
 endclass
 typedef mailbox #(Xaction) mbx;
 mbx mb = new ();

 initial
 begin: t
   Xaction xaction;
   int mb_size;

   for (int i=0; i<5; i++)
   begin
     xaction = new;
     xaction.addr = 3'b111;
     $display("BEFORE:: Addr = %h",
                xaction.addr);
     mb.put(xaction);
   end
   mb_size = mb.num();
   for (int i=0; i<mb_size; i++)
   begin: dis_l
       Xaction d_x;
       mb.get(d_x);
       $display("Addr = %h",
                  d_x.addr);
    end: dis_l
  end: t
endmodule: mailbox_ex
```

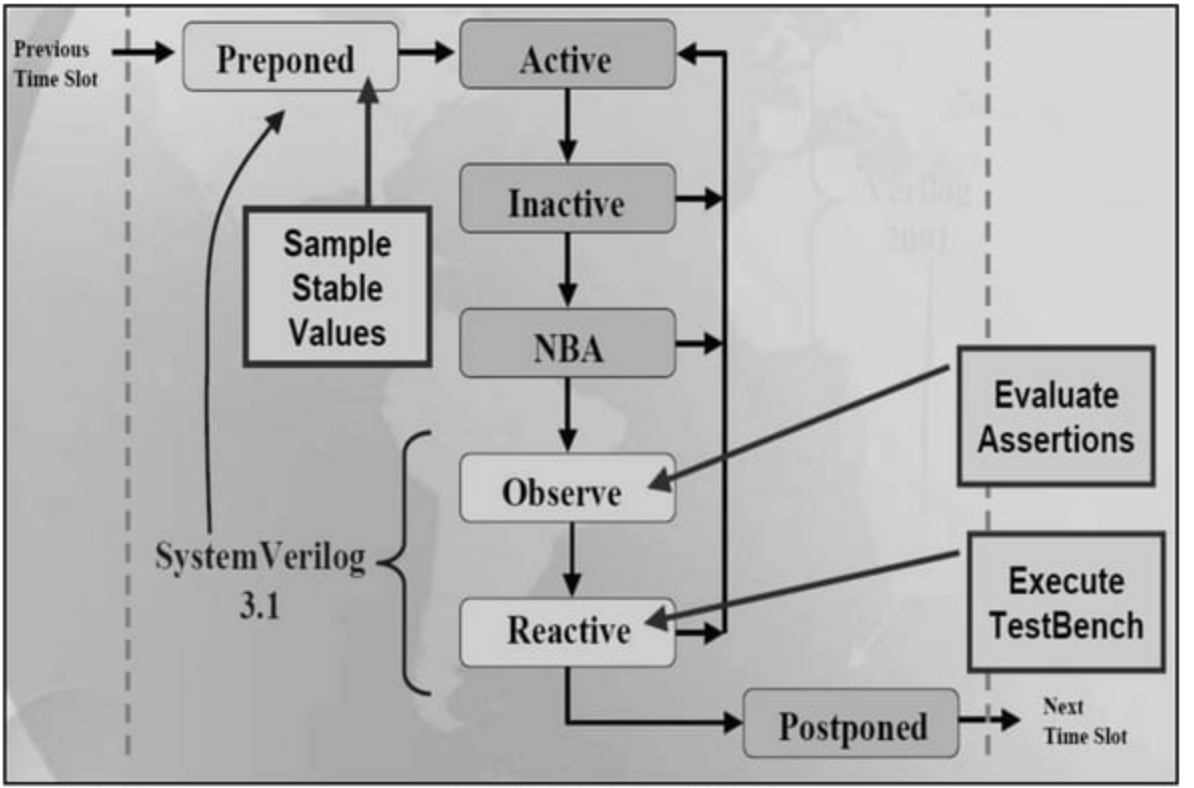
www.verificationexcellence.in

- For more Course Resources, Tutorials see
- http://verificationexcellence.in/

System Verilog Assertions

System Verilog Scheduling

System Verilog Enhanced Scheduling



Standard Verification Methodologies

Why do we need Standard Methodology?

- Traditional methods doesn't scale up or enable re-use across verification for complex design
- Verification is increasingly becoming critical and complex
  - Building constrained random test benches is complex and time consuming
  - Projects demand verification engineers to divide and conquer
- Following needs to be addressed for efficiency
  - Automation
  - Abstraction
  - Re-use
  - Interoparability
  - Quality

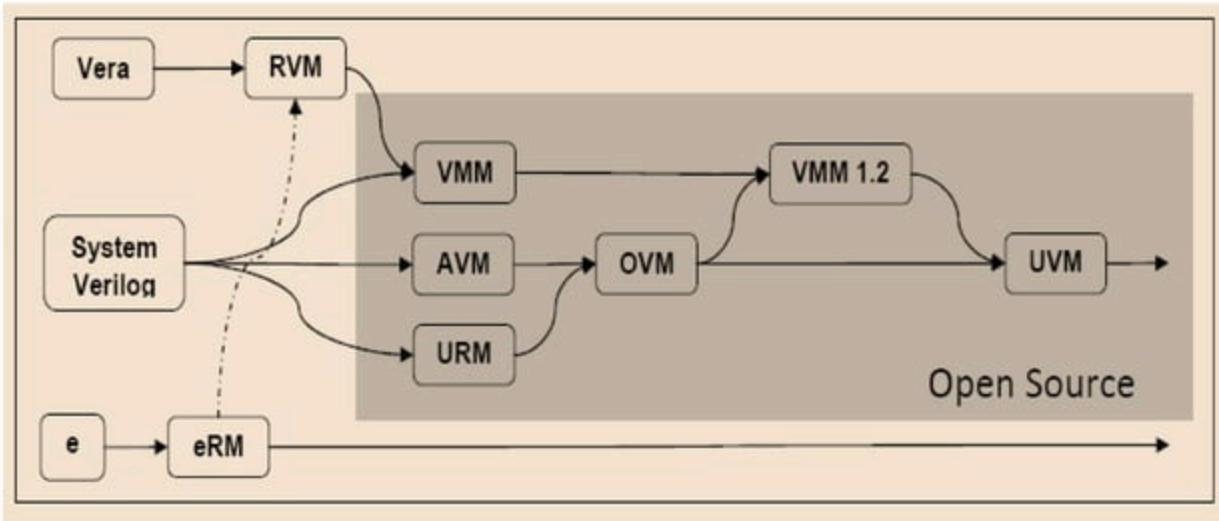
What must a methodology Provide?

- Standard to enable re-use
  - Abstraction/Project/Company/Industry wide re-use
- Layered approach to enable a division of skills and development
  - Developing verification IP
  - Re-using and configuring existing verification IP
  - Separation of Stimulus from Testbench Structure
  - Writing of new code
    - Assertions and/or coverage points
    - Writing or re-using checkers
    - Writing or re-using test cases

What must a methodology Provide?

- A consistent approach
  - Naming conventions
  - Verification IP configuration
    - Defining the number of agent
    - How they connect to the DUT
    - Whether an agent is ACTIVE or PASSIVE
  - Well-defined generation and simulation phases
    - Build, connect, pre-run, run, post-run
- The power to find bugs fast
  - Faster development of Testbench
  - Good control on stimulus, observability and debug ability
- Vendor/Tool independence
- Verification Management - Planning, progress and completion

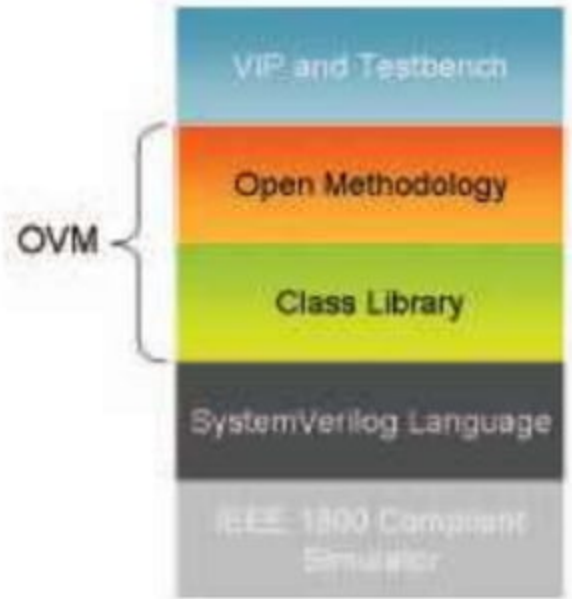
History of Verification Methodologies



What is OVM/UVM ?

- OVM - Open Verification Methodology
  - Backward compatible with AVM and URM
- UVM - Universal Verification Methodology
  - Accellera standard
  - SystemVerilog UVM Base Class Library (BCL)
  - Near-backward compatible with OVM
- Both are Open source (Apache licence)
- Test benches for (System)Verilog / VHDL / SystemC designs
- SystemVerilog class library

OVM/UVM Conceptual View



Highlights of OVM and UVM

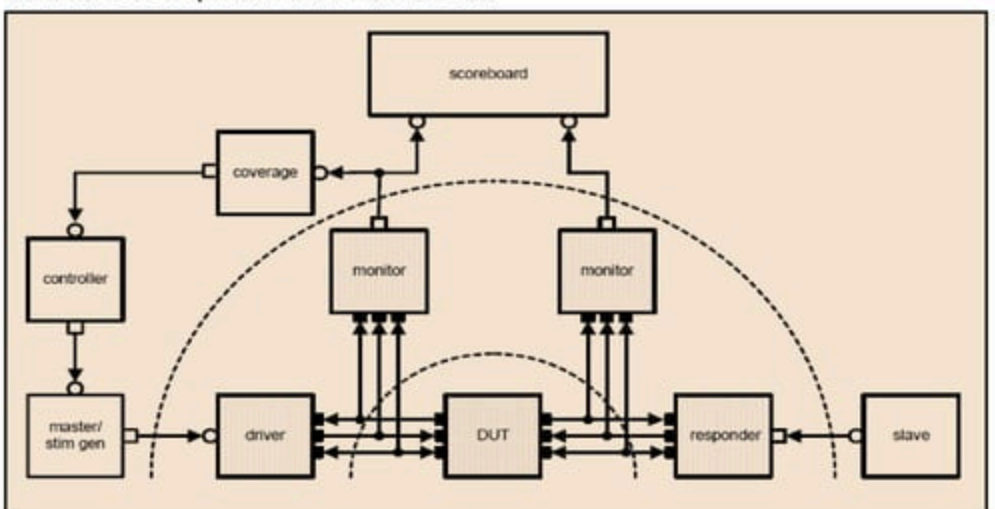
- Constrained random, coverage-driven verification
- Configurable, flexible, test benches
- Verification IP reuse
- Separation of tests/stimulus from test bench
- Transaction-level communication (TLM)
- Layered sequential stimulus
- Standardized messaging
- Register layer (Newly added in UVM)
  - Enables programming and verifying registers in a consistent and efficient manner

UVM – OVM - Differences

- UVM is based on OVM 2.1.1
  - The deprecated features from OVM were removed in UVM (deprecated.txt file in the OVM install area).
  - The URM and AVM compatibility layers were removed from UVM.
- Updates in UVM
  - Enhancements to the OVM callback facility, including a new message catching facility.
  - Enhancements to the OVM objection mechanism.
    - These enhancements introduce some minor backward incompatibilities to the OVM callback facility.
  - Few capabiilties donated from VMM (not sure what?)
  - Other than this – it is a blind change from ovm_* to uvm_* for the whole library

Layered Test bench Architecture

- Structured Test benches are important for complex DUT Verification
  - Better maintainability
  - Better reusability
  - Parallel Development and Verification



Layered Test bench Architecture

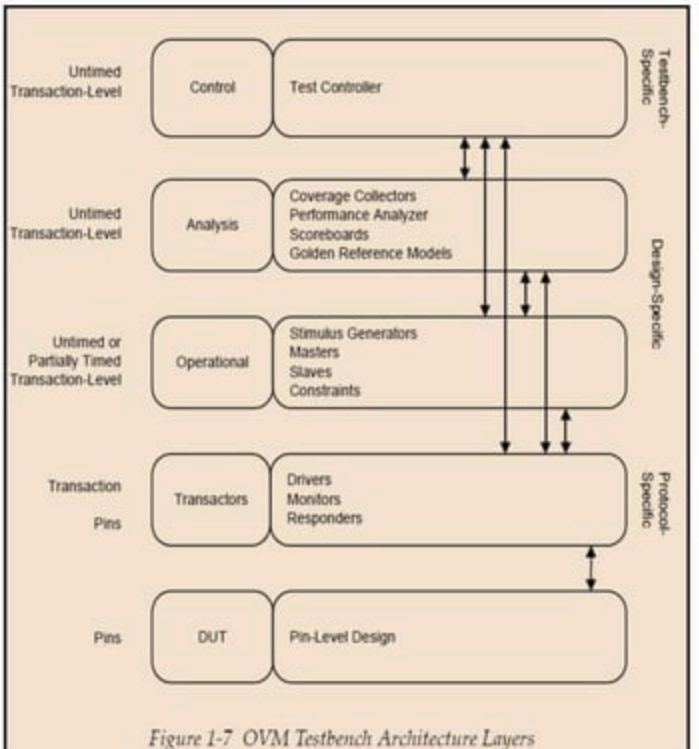


Figure 1-7 OVM Testbench Architecture Layers

- Bottom layer – Pin level interface
- Next layer abstracts to transactions
- All further layers above work on transactions

Transactors

- Components that converts stream of transaction to pin level interface and vice versa
  - Monitor
    - Passive components that watches the pin toggles and converts them to a stream of transactions
  - Driver
    - Converts a stream of transactions to pin level activity
  - Respondent
    - Similar to a driver but responds to an activity rather than initiating activity

Operational Components

- Transactional components that provides everything needed for DUT to operate
  - Stimulus Generator
    - Creates stream of transactions to stimulate the DUT. Can be random, directed or pseudo-random
  - Master
    - Bi-directional component that can send requests and can receive responses
  - Slave
    - Bi-directional component that responds to requests by returning with responses

Analysis and Controller Components

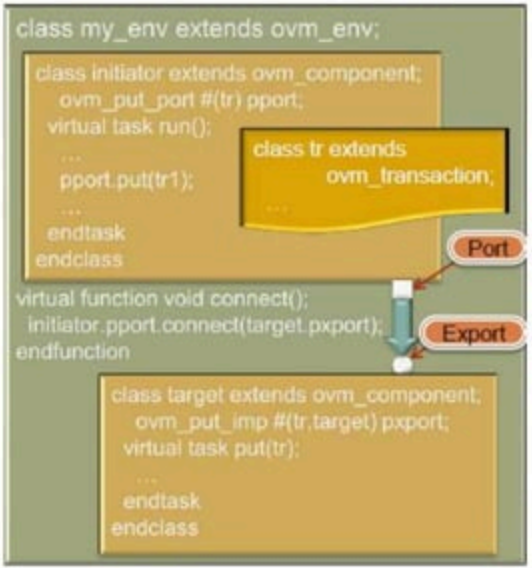
- Analysis components receive information about activities in the testbench and use that information for determination of correctness or completion
  - Scoreboards
    - They receive information about what's going into and out of the DUT and analyze those for the correctness of DUT
  - Coverage collector
    - The count streams of transactions and different aspects to transactions and help in determining the completeness of verification
- Controller forms the main thread of the test
  - They receive and send information from various testbench components to control the test
  - They control the start and stop of stimulus generators

Transaction Level Modelling

- Transaction-level models consist of multiple processes communicating with each other by sending transactions back and forth through channels.
  - Abstracts time
    - Reduces the number of activation of processes
    - RTL evaluation of nets/signals vs evaluation of transactions
  - Abstracts data
    - Form transaction objects instead of bit by bit details
  - Abstracts function
    - Implement functionality with functions/processes rather than with real registers or circuits

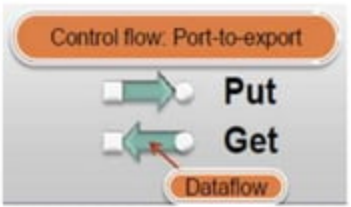
Understanding TLM - OVM/UVM

- TLM is the basis for modularity and re-use in OVM/UVM
- TLM is all about *communication* through *method calls*
  - A TLM *port* specifies the "API" to be used
  - A TLM *export* supplies the implementation of the methods
- Connections are between ports/exports, not components
- Transactions are *objects*
- Ports & exports are parameterized by the transaction type being communicated



TLM interfaces supported

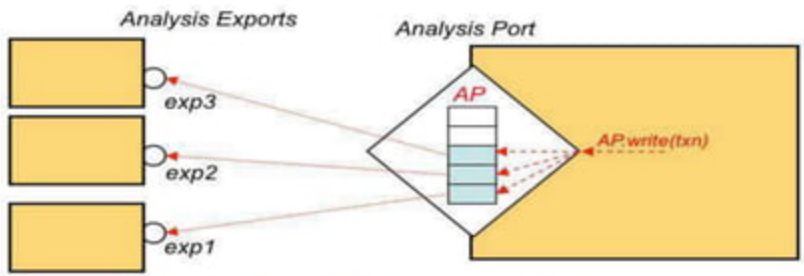
- Unidirectional
  - Blocking (Tasks)
    - put
    - get/peek
  - Nonblocking (functions)
    - try_put
    - try_get/try_peek
    - write



- Bi directional
  - Master
    - put_request
    - get_response
  - Slave
    - get_request
    - put_response
  - Transport
    - transport
    - nb_transport (function)
  - Sequence/Driver
    - get_request
    - put_response

Analysis Ports

- Analysis ports support 1:many connections
- Meets “observer pattern” where observers register with a single source of information
- Used by coverage collectors and score boards
- write() needs to be non-blocking



www.verificationexcellence.in

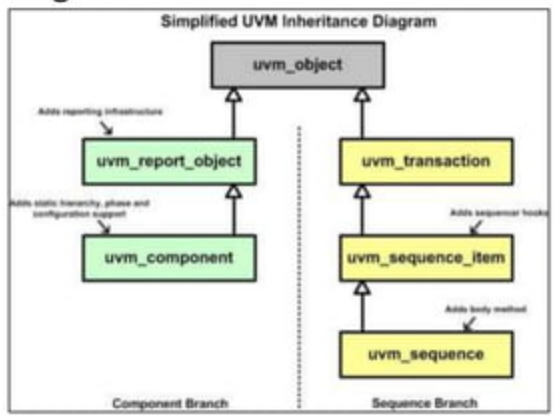
- For more Course Resources, Tutorials see
- http://verificationexcellence.in/

UVM Testbench Hierarchy

- Testbenches are built from classes derived from uvm_component class
- Top level class is a test class that instantiates “env” class which instantiates a series of components
- Modular to facilitate the reuse of groups of verification components
  - Across projects (horizontal reuse)
  - Across levels of integration (veritical reuse)
- Two main collective component types
  - the env (short for environment)
  - the agent

UVM Component and Sequences

- Structural components of Testbench and Stimulus generation are kept separate.
- All structural components of testbench are derived from uvm_component base class
  - Creates a static hierarchy
- All transient objects are derived from uvm_sequence base class which are created, used and garbage collected when de-referenced

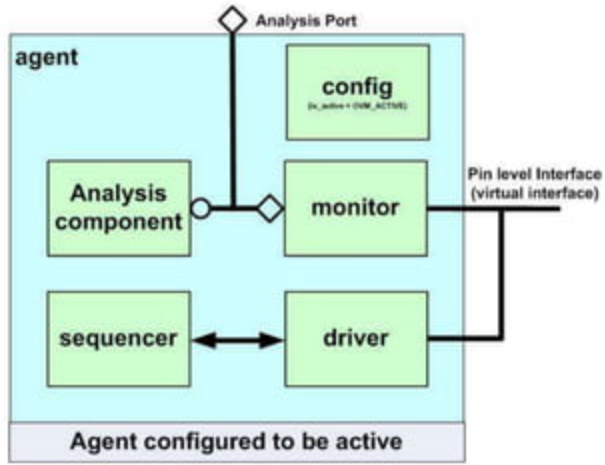


Available UVM components

Class	Description	Contains sub-components?
uvm_driver	Adds sequence communcation sub-components, used with the uvm_sequencer	Yes
uvm_sequencer	Adds sequence communcation sub-components, used with the uvm_driver	Yes
uvm_subscriber	A wrapper around an uvm_analysis_export	Yes
uvm_env	Container for the verification components surrounding a DUT, or other envs surrounding a (sub)system	No
uvm_test	Used for the top level test class	No
uvm_monitor	Used for monitors	No
uvm_scoreboard	Used for scoreboards	No
uvm_agent	Used as the agent container class	No

UVM Agent

- UVM agent
  - collection of a group of uvm_components focused around a specific pin-level interface
  - Multiple agents can be used for multiple DUT interfaces which has different protocol
  - Agents can be reused for similar interfaces

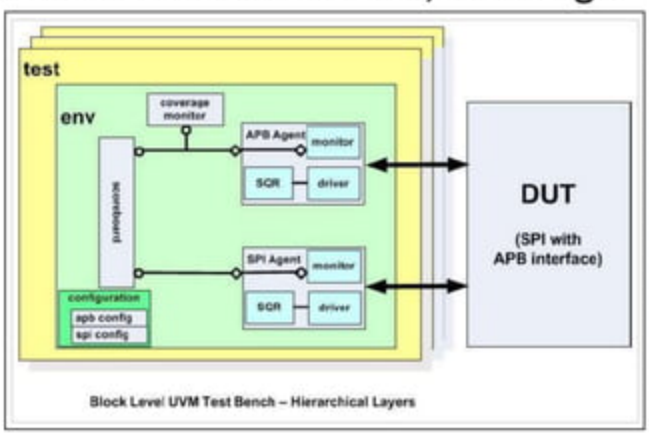


Agent Components

- **Driver** – Converts data inside a series of sequence_items into pin level transactions.
- **Sequencer** - Routes sequence_items from a sequence where they are generated to/from a driver.
- **Monitor** - Observes pin level activity and converts its observations into sequence_items which are sent to analysis components
- **Configuration object** – Contains information about agent which determines what it does and how it is built and connected.

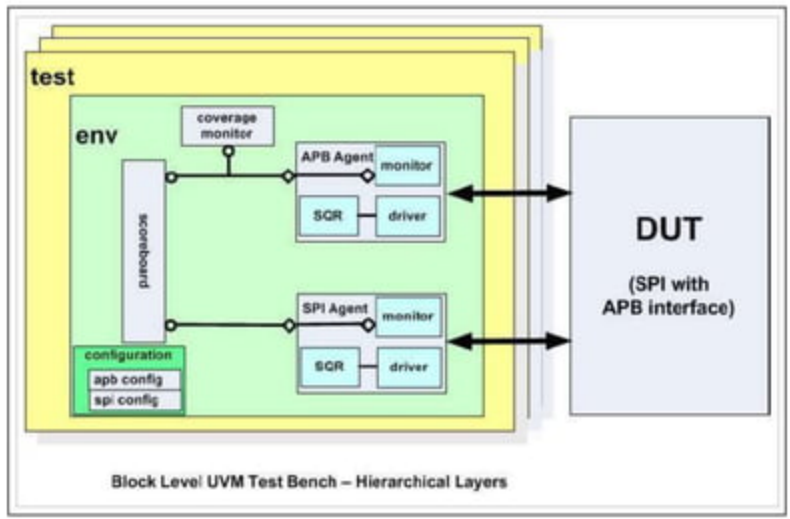
UVM Env

- Collection of UVM Agents
  - Allows interface level reuse
- Has its own configuration object
  - eg: can control no of agents, env.config can set agent.config
- Can have analysis components like scoreboard, coverage monitor
- Multiple block level env classes can be instantiated to have top level env



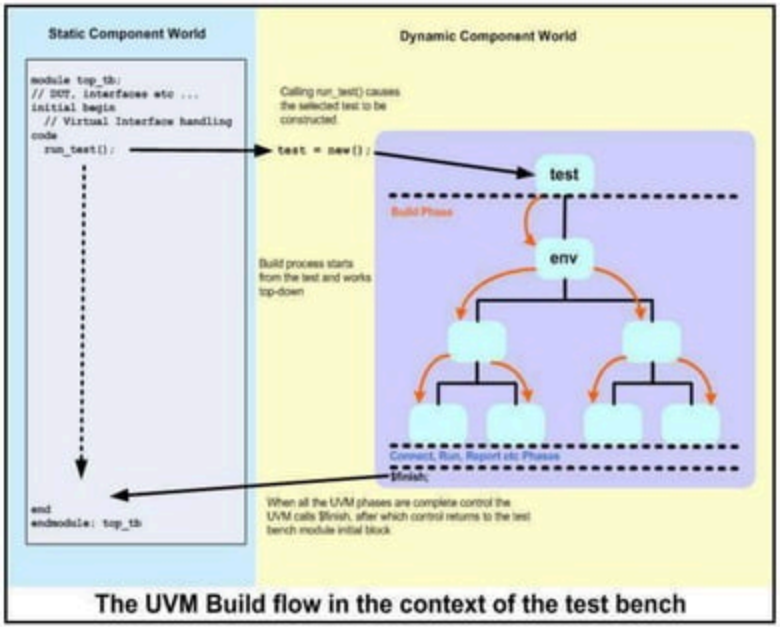
UVM Test

- Test is the top level class that instantiatiates ENV, configures the testbench and intiates construction
- Individual tests derive from uvm_test
  - Each test case instantiates uvm_env and configures them
- Testbench activated with a call to run_test() which starts build phases



Testbench Build

- All uvm_component_class objects in Testbench hierarchy constructed top down
  - Build() method in uvm_test gets called first and then propagates down

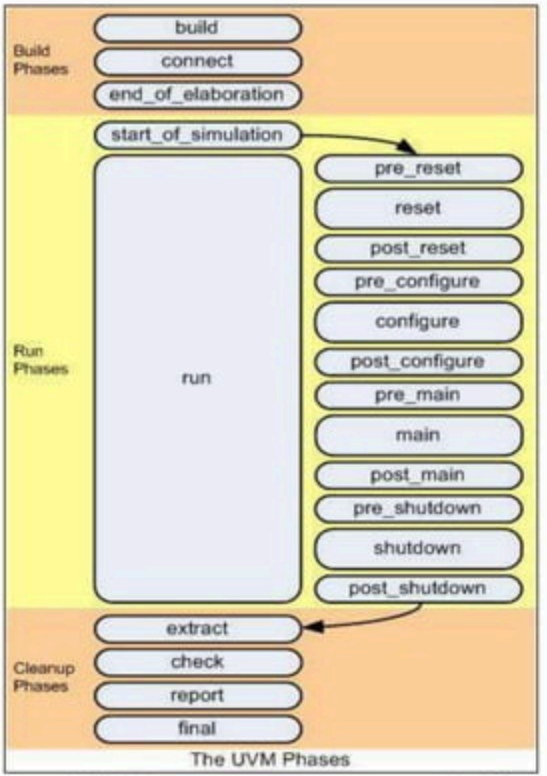


The UVM Build flow in the context of the test bench

UVM/OVM Simulation Phases

- When using classes, you need to manage environment creation at run-time
- Test execution is divided to phases
  - Configuration, testbench creation, run-time, check, etc
- Unique tasks are performed in each simulation phase
  - Set-up activities are performed during "testbench creation" while expected results may be addressed in "check"
  - Phases run in order – next phase does not begin until previous phase is complete
- OVM/UVM provides set of standard phases enabling VIP plug&play

UVM Phases



The UVM Phases

- Build Phases
  - Build Top-Level Testbench Topology
  - Connect environment topology
  - Post-elaboration activity (e.g. print topology)
- Run Phases
  - Run-time execution of test
  - All phases except run() execute in zero time
  - Lots of sub-phases not used really
- Cleanup Phases
  - Gathers details on the final DUT state
  - Processes and checks the simulation results.
  - Simulation results analysis and reporting

Run Phases

- **start_of_simulation()**
  - occurs before the time consuming part of the testbench begins. Called in bottom up order
- **run()**
  - stimulus generation and checking activities
  - all uvm_component run tasks are executed in parallel.
- **pre_reset()**
  - Its purpose is to take care of any activity that should occur before reset eg: waiting for a power good signal to go high
- **reset()**
  - for DUT or interface specific reset behaviour.
- **post_reset() -**
  - Intended for any activity required immediately following reset.
- **pre_configure()**
  - Intended for anything that is required to prepare for the DUT's configuration process after reset completion.
- **configure()**
  - Used to program the DUT and any memories in the testbench so that it is ready for the start of the test.

Run Phases

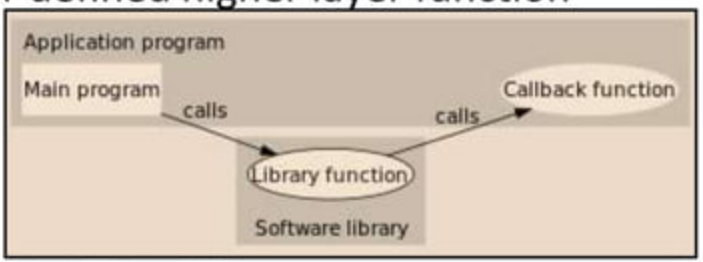
- **post_configure()**
  - Used to wait for the effects of configuration to propagate through the DUT.
- **pre_main()**
  - Used to ensure that all required components are ready to start generating stimulus.
- **main()**
  - This is where the stimulus specified by the test case is generated and applied to the DUT. It completes when either all stimulus is exhausted or a timeout occurs.
- **post_main()**
  - Used to take care of any finalization of the main phase.
- **pre_shutdown()**
  - This phase is a buffer for any DUT stimulus that needs to take place before the shutdown phase.
- **Shutdown()**
  - Used to ensure that the effects of the stimulus generated during the main phase have propagated through the DUT and that any resultant data has drained away.
- **post_shutdown()**
  - Perform any final activities before exiting the active simulation phases.

UVM Factory

- UVM Factory
  - Allows an object of one type to be substituted with an object of derived type without testbench edits/re-compile
  - Works on the concept of polymorphism which allows a derived class object to be accessed using a base class type handle
- All objects should be registered with factory class
  - `ovm_object_utils (for sequences) and `ovm_component_utils (for components)
  - Use ::create() method of factory instead of calling new()
- Two types of override possible
  - Type overrides
    - Every time a component class type is created in a testbench hierarchy, a substitute type is created in its place. Applies to all instances of that component type
  - Instance override
    - A specific component instance is overridden by specifying its position in the uvm component hierarchy

Callbacks

- Like the factory, callbacks are a way to affect an existing component from outside
  - The SystemVerilog language includes built-in callbacks e.g. post_randomize(), pre_body()
- A call back function can be defined as a virtual function in the base class and implemented in the derived class as needed
- Similar to how in computer programming – how a low level software can call a user defined higher layer function



UVM Configuration

- Test benches need to be as configurable as possible for re-use
  - Each component can have a config object
  - Global config objects are also possible
- Several variables can be configurable and test can control runtime
  - e.g. for loop limits, randomization weightages, coverage bins etc.

- UVM config database takes care of scope and storage of these config objects

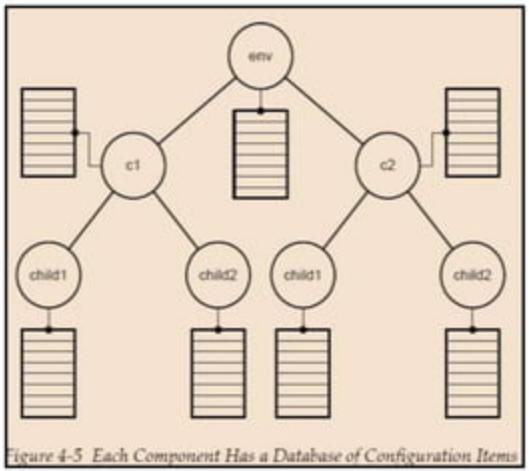


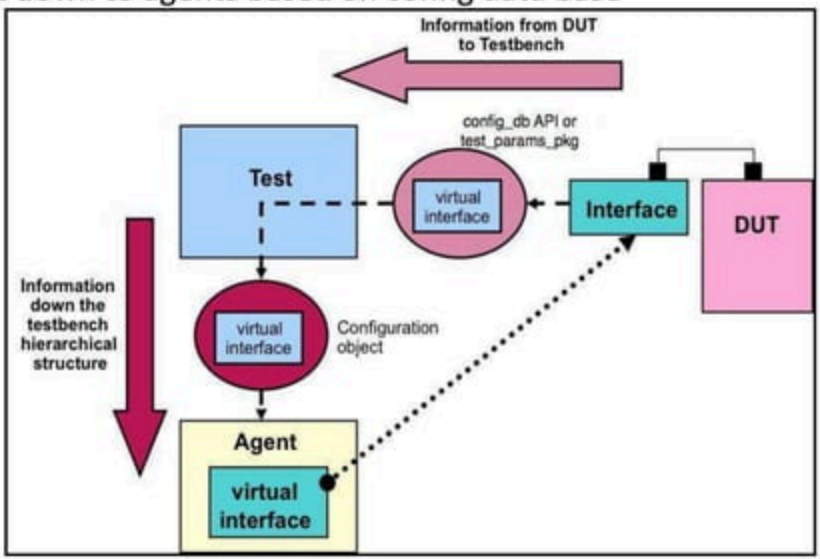
Figure 4-5 Each Component Has a Database of Configuration Items

UVM Configuration

- uvm_config_db class is recommended way of accessing config across components
  - ::set* and ::get* methods available
- Two typical uses with uvm_config_db
  - Passing virtual interfaces from DUT to test
    - In module top() - add the physical interface into config db using config_db::set() method
    - In class uvm_test() – get the handle to interface using config_db::get() and pass down to agents
  - Passing configuration classes down through TB

DUT – TB connection

- DUT is in static world and TB is in dynamic world
- For modularity and re-use – DUT should provide information of interfaces to testbench without knowing how it is configured
  - Test to propagate it down to agents based on config data base

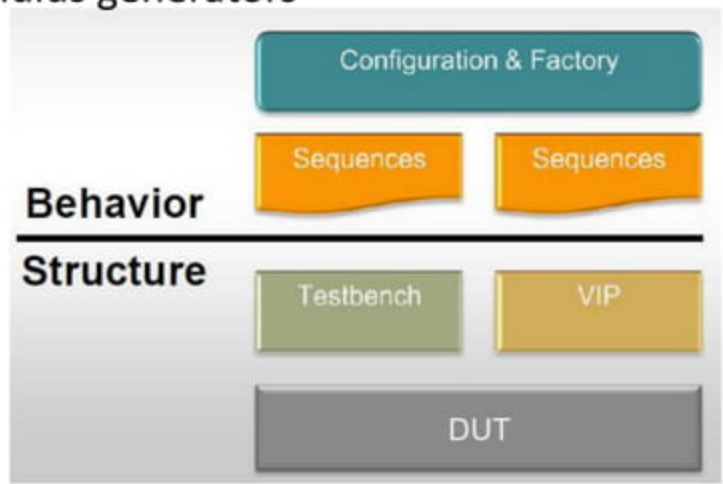


End of Test Mechanism - Objections

- End of Test
  - Happens when all time consuming phases end
  - A phase end happens when there are no pending objections to that phase
- uvm_objection class
  - Provides a shared counter between all participating components and sequences
  - Each participant can “raise” (increase) or “drop” (decrease) objection
  - Used in run() phase of UVM to determine end of phase when an “all dropped” (zero count) is reached
- Methods
  - ::raise_objection()
  - ::drop_objection()

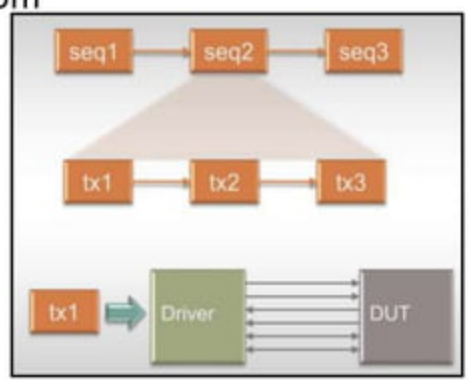
Sequences - Seperating Stimulus from Testbench

- Key to reusability
  - Separate Behavior from Structure
- Sequences provide a highly modular and flexible means for building complex stimulus generators



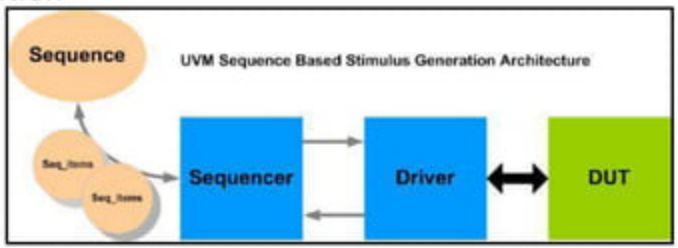
Layered Sequential Stimulus

- Every stimulus drives some transactions as pins on DUT
  - eg: a read or a write transaction to a DUT
- A group of transactions can be combined to form a sequence
  - e.g. read-modify-write
  - Can be a deterministic sequence
  - Can be a random or constrained random
- A test will define what sequences and the order of them
- Enables re-use at different levels of verification



UVM Sequences - Overview

- A sequencer controls the generation of random stimulus by executing sequences
- A sequence captures meaningful streams of transactions
  - A simple sequence is a random transaction generator
  - A more complex sequence can contain timing, additional constraints, parameters
- Sequences:
  - Allow reactive generation –react to DUT
  - Have many built-in capabilities like interrupt support, arbitration schemes, automatic factory support, etc
  - Can be nested inside other sequences
  - Are reusable at higher levels



UVM Sequences

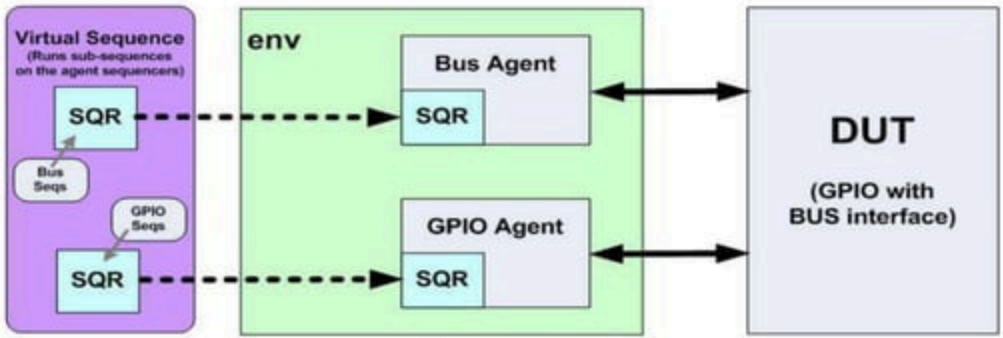
- Sequence is a transient object with limited life time unlike a uvm_component
  - Each sequence will have to implement a task called body()
  - This can be implemented to create other sequences or to generate sequence_item and send to driver
- Sequencer is an intermediate component that implements communication channels and arbitration mechanisms between sequence and driver
- Sequence item
  - Data object that contains all the needs of a driver to driver on pin level
  - Most randomization done on this object
  - Should implement all of the virtual methods like copy/clone/compare/print etc

Sequence Execution Flow

- Call *uvm_sequence::start()* to start sequence execution
  - Will internally call the body() method
  - The body() can implement what all sequence_items to be send to driver or whether to spawn other sequence etc..
- Controlling multiple sequences to a single driver
  - Sequencer has 5 arbitration schemes that can be configured
  - Sequence id field can be used for correlating response from drivers to a sequence.
  - Methods like grab or lock are available if a sequence need exclusive access to driver
- Controlled stimulus generation on multiple drivers connected to different interface
  - Use a Virtual sequence which can start sub-sequences

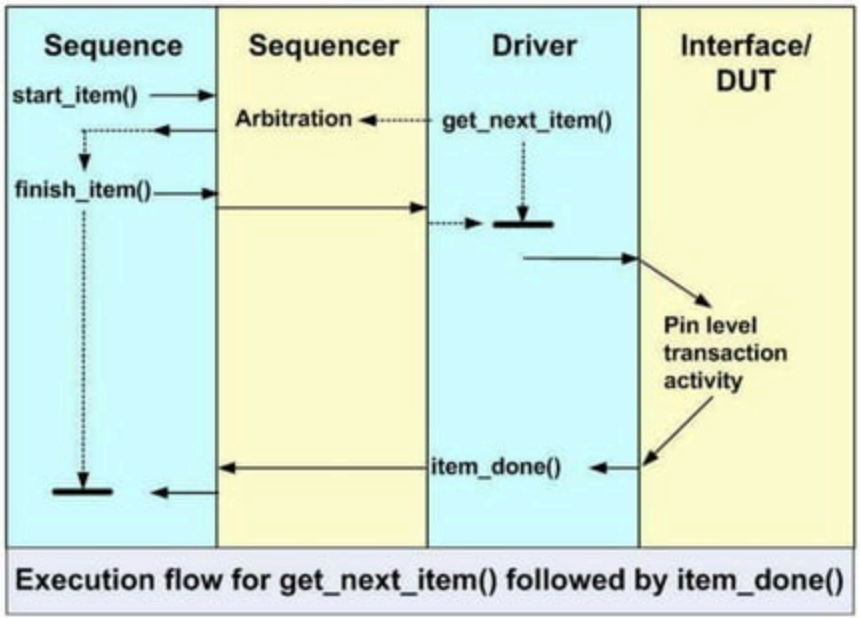
Virtual Sequences

- A virtual sequence is a sequence that can start sub-sequences on multiple sequencers in different agents.
  - Also called "co-ordinated sequence"
- In below e.g. you might want a specific bus sequence happening at same time when a specific GPIO sequence is happening on a different interface in a controlled manner



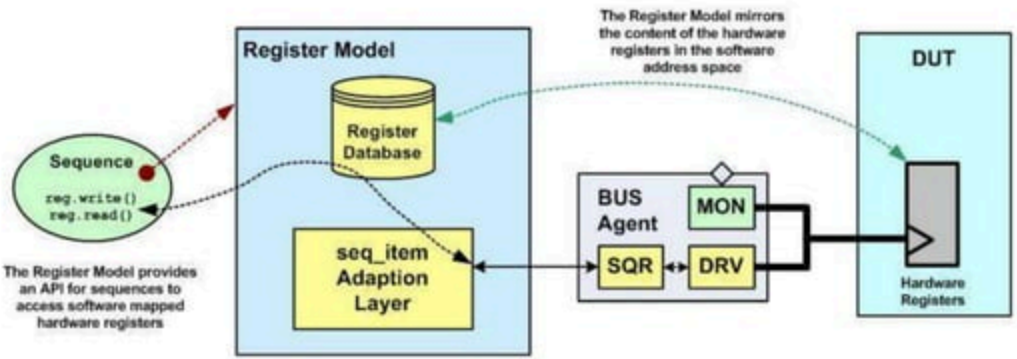
The Virtual Sequence

Sequence-Driver API Calls



Register Abstraction Layer

- Abstracts a common reference specification for all registers
  - To be shared across design/verification/Software teams
  - Can be created using a generator application or hand coded



UVM Register Model Functional Overview

Thank You

- For more courses and resources follow
  - verificationexcellence.in
- Follow on Face book
  - https://www.facebook.com/verificationexcellence
- Follow on Twitter
  - https://twitter.com/ramdas2m